IIT GANDHINAGAR • SSRP 2026 • PROJECT 7

Real-Time EMG Signal Processing & Biceps-Controlled VR Weightlifting

The Definitive Mathematical, Architectural & Pedagogical Master Reference: From Biological Action Potentials to Discrete-Time Digital Filters, Calibration Algorithms, and UDP Telemetry

Surface Electromyography | 4 Digital Filters Compared | Two-Point Calibration | Low-Latency UDP Socket | Unity VR Biofeedback

Project:	SSRP Project 7 • Real-Time Human-Machine VR Interfaces
Institution:	Indian Institute of Technology Gandhinagar (IITGN), Gujarat, India
Student Team:	Raima Ghosh (IIR) • Kunal Kashyap (FIR) • Chandan Kumar (FFT) • Krishna Patel (DIFF)
Hardware & Stack:	GLNT BioRadio Wireless System, Python 3.10 (SciPy / NumPy), C# Unity Engine
Theoretical Basis:	B.P. Lathi, *Linear Systems and Signals* (3rd Edition, Oxford University Press)
Document Purpose:	Exhaustive 35+ Page Technical Reference Manual & Pedagogical Blueprint

DOCUMENT BLUEPRINT

Executive Summary & Master Table of Contents

An architectural roadmap bridging human electrophysiology, rigorous linear systems signal processing, and low-latency virtual reality cybernetics.

Executive Project Overview

This master guide documents the research, mathematical modeling, digital implementation, and empirical validation conducted during **Summer Student Research Programme (SSRP 2026) Project 7** at the **Indian Institute of Technology Gandhinagar (IITGN)**. The primary objective of this project is to construct an end-to-end, sub-100-millisecond cybernetic link connecting human skeletal muscle contractions to a real-time 3D Virtual Reality (VR) weightlifting simulator.

Controlling an interactive simulation directly with human muscles requires translating faint, stochastic, microvolt-level electrophysiological potentials recorded on the skin surface into a clean, rock-solid, normalized physical activation number between 0.0 and 1.0. This endeavor faces four formidable hurdles:

- **Vast Noise Contamination:** The raw signal is corrupted by electromagnetic 50 Hz power-line interference, low-frequency motion artifacts from skin shearing, and high-frequency thermal/radio noise.
- **The Zero-Sum Nature of Muscle Signals:** Because raw action potentials are biphasic electrical dipoles, raw electromyographic signals have an average voltage of zero regardless of whether the athlete is resting or curling a heavy dumbbell.
- **Real-Time Causality & Streaming Latency:** Unlike offline biomedical research where non-causal zero-phase filters (such as forward-backward filtering) can process entire static files, an interactive VR game demands streaming sample-by-sample or block-by-block processing with under 50 ms group delay to prevent cybersickness and motor dissociation.
- **The Isometric Baseline Shift ("The 6 kg Mystery"):** When an athlete grasps a physical 6 kg weight with the elbow extended, tonic isometric stabilizing co-contractions produce sustained high-amplitude electrical activity, erroneously causing naive VR avatars to lift the dumbbell before any active curl begins.

To solve these challenges with academic rigor and engineering excellence, four parallel digital filtering methodologies were investigated, authored by student researchers: **Raima Ghosh** (4th-order Butterworth IIR via Second-Order Sections), **Kunal Kashyap** (101-tap linear-phase FIR via Hamming Windowing), **Chandan Kumar** (Fast Fourier Transform Block Spectral Masking), and **Krishna Patel** (First-Order Difference Operator). These filters are paired with a causal moving-window Root Mean Square (RMS) envelope detector, a robust Two-Point Calibration algorithm, a non-linear dead zone, and continuous exponential smoothing, streaming telemetry over a dedicated local UDP socket at 50 Hz.

Master Table of Contents

PART I: THE HUMAN BODY AS AN ELECTRICAL MACHINE

Chapter 1: Introduction to Bioelectricity and Surface Electromyography (sEMG) Ch. 1

1.1 Muscle as an Electric Motor • 1.2 Depolarization & Action Potentials • 1.3 sEMG vs Intramuscular • 1.4 Biceps Biomechanics • 1.5 Amplitudes & Spectra

Chapter 2: Data Collection Hardware — The BioRadio Acquisition System Ch. 2

2.1 Electrode-Skin Interface • 2.2 Differential Recording & Ground • 2.3 Instrumentation Amplifier & CMRR • 2.4 Analog-to-Digital Conversion • 2.5 Nyquist Theorem • 2.6 Bluetooth Stream

PART II: THE NOISE MONSTERS AND WHY FILTERING IS ESSENTIAL

Chapter 3: Sources of Noise and Contamination in Electromyographic Signals Ch. 3

3.1 50 Hz Mains Hum • 3.2 Motion Artifacts (<20 Hz) • 3.3 High-Frequency Noise (>450 Hz) • 3.4 Muscle Crosstalk • 3.5 Baseline Drift • 3.6 The Zero-Sum Trap

Chapter 4: Fundamentals of Discrete-Time Digital Signal Processing (DSP) Ch. 4

4.1 Linear Time-Invariant Systems • 4.2 The z-Transform & Transfer Function • 4.3 Practical Filter Specifications • 4.4 Bandpass Filtering • 4.5 Notch Filtering • 4.6 Causality & States

PART III: THE FOUR FILTER SUPERHEROES — DEEP TECHNICAL ANALYSIS

Chapter 5: Infinite Impulse Response (IIR) Filter — Raima's Method Ch. 5

5.1 Recursive Feedback • 5.2 Butterworth Filter Design • 5.3 Second-Order Sections (SOS) • 5.4 50 Hz IIR Notch • 5.5 Streaming State Management • 5.6 Latency Analysis

Chapter 6: Finite Impulse Response (FIR) Filter — Kunal's Method Ch. 6

6.1 Non-Recursive FIR Theory • 6.2 Linear Phase & Waveform Fidelity • 6.3 Window Method & Hamming Taps • 6.4 Python Implementation • 6.5 Deep IIR vs FIR Comparison

Chapter 7: Fast Fourier Transform (FFT) Filter — Chandan's Method Ch. 7

7.1 Time-Frequency Duality • 7.2 Discrete Fourier Transform • 7.3 Cooley-Tukey Radix-2 • 7.4 Block Spectral Masking • 7.5 Circular Convolution & Window Leakage

Chapter 8: First-Order Difference (DIFF) Filter — Krishna's Method Ch. 8

8.1 Differentiation as High-Pass • 8.2 Frequency Response • 8.3 Onset Detection vs Noise Boost • 8.4 Streaming Boundary Preservation

PART IV: FROM FILTERED SIGNAL TO MUSCLE POWER NUMBER

Chapter 9: Rectification, Root Mean Square (RMS), and the Linear Envelope Ch. 9

9.1 Full-Wave Rectification • 9.2 The Mathematical Anatomy of RMS • 9.3 Moving-Window RMS Buffer • 9.4 Linear Envelope Dynamics • 9.5 Window Trade-offs

Chapter 10: Two-Point Calibration — Solving the 6 kg Mystery Ch. 10

10.1 The 6 kg Paradox • 10.2 Isometric Contraction Biomechanics • 10.3 Two-Point Calibration Algorithm • 10.4 Percentile Robustness • 10.5 Dead Zone • 10.6 Exponential Smoothing

PART V: REAL-TIME COMMUNICATION AND DATA LOGGING

Chapter 11: UDP Network Telemetry & Protocol Architecture Ch. 11

11.1 Transport Layer: TCP vs UDP • 11.2 Head-of-Line Blocking • 11.3 Localhost Socket Implementation • 11.4 Telemetry JSON Payload Schema • 11.5 Packet Drop Resilience

Chapter 12: The Python Bridge Application Architecture Ch. 12

12.1 System Integration • 12.2 BioRadioSDK via Pythonnet • 12.3 Parallel Processing Engine • 12.4 Real-Time GUI Visualization • 12.5 12-Column CSV Scientific Logging

Chapter 13: The Complete Signal Processing Pipeline & Latency Budget Ch. 13

13.1 End-to-End Cybernetic Journey • 13.2 Cumulative Latency Budget • 13.3 Four-Method Master Scorecard • 13.4 Conclusions & Future Work

PART I: THE HUMAN BODY AS AN ELECTRICAL MACHINE • CHAPTER 1

Introduction to Bioelectricity and Surface Electromyography

Exploring the electrophysiological mechanisms of human muscle contraction, the genesis of motor unit action potentials, and why the biceps brachii serves as an ideal natural transducer for cybernetic virtual reality interfaces.

1.1 The Muscle as a Biological Electric Motor

To build a system that allows a computer or a Virtual Reality (VR) headset to read a human's physical intentions, we must first recognize a fundamental biological truth: **the human muscular system is an electro-chemical machine**. Every time you decide to move your arm, catch a ball, or curl a barbell, your thoughts are converted into literal electrical impulses that race down your nervous system at speeds exceeding 100 meters per second.

The command originates in the primary motor cortex of the brain. Pyramidal neurons fire action potentials down the corticospinal tract through the spinal cord. In the anterior horn of the spinal cord, these signals synapse with specialized biological control units known as **alpha motor neurons (α-motor neurons)**. An individual motor neuron extends a long axon that travels out through peripheral nerves directly into a specific muscle, such as the *biceps brachii*.

Inside the muscle belly, the axon branches out into multiple terminals, each forming a specialized biochemical junction called a **neuromuscular junction (NMJ)** with an individual muscle fiber. The functional atomic unit of skeletal muscle contraction is formally termed the **Motor Unit (MU)**.

> 🔬 **Technical Fact & Definition: The Motor Unit Definition**
>
> A **Motor Unit** is defined as a single alpha motor neuron and all the individual skeletal muscle fibers that it innervates. When the motor neuron fires an electrical impulse, every single muscle fiber within that motor unit contracts synchronously in an all-or-none fashion. In fine-motor muscles (like the eyes or vocal cords), a motor unit may contain as few as 5 to 10 muscle fibers. In large postural and locomotion muscles (like the biceps, quadriceps, or gastrocnemius), a single motor unit may command between 500 and 2,000 muscle fibers simultaneously!

> 💡 **Intuitive Analogy: The Orchestra Conductor and Musicians**
>
> Imagine an enormous symphony orchestra with 1,000 violinists. The **brain** is the master conductor standing on stage. The **motor neurons** are the section leaders, each responsible for instructing a cluster of 50 violinists. The individual **muscle fibers** are the violin players themselves.
>
> When the conductor wants a quiet, gentle melody (lifting a feather), only two or three section leaders give the signal, and a few violins play softly. When the conductor demands an earth-shattering crescendo (lifting a 20 kg dumbbell), all section leaders shout simultaneously, and all 1,000 violinists draw their bows with maximum vigor. The resulting sound wave hitting your ears is the acoustic equivalent of the electromyographic signal picking up electrical ripples through the skin!

Henneman's Size Principle of Motor Unit Recruitment

Muscles do not turn on like a simple household light switch (0% or 100%). Instead, the central nervous system grades the force of contraction through two coordinated mechanisms: **spatial recruitment** (varying the number of active motor units) and **rate coding** (varying the firing frequency of each active motor unit, typically between 8 Hz and 50 Hz).

The orderly recruitment of motor units obeys **Henneman's Size Principle**. Small motor neurons, which possess small cell bodies and low electrical thresholds, innervate fatigue-resistant *Type I (slow-twitch)* muscle fibers. As the required mechanical load increases, the nervous system progressively recruits larger motor neurons with higher electrical thresholds, which innervate powerful, fast-fatiguing *Type IIa and Type IIx (fast-twitch)* muscle fibers. This physiological fact is vital when designing VR weightlifting algorithms: lifting no load recruits small, gentle motor units, whereas curling a 6 kg dumbbell recruits large, high-voltage motor units with distinct spectral signatures.

1.2 Action Potentials, Depolarization, and Repolarization

At the microscopic level, how does a muscle fiber actually create an electrical voltage? All living animal cells maintain an electrical potential difference across their outer cellular membrane (the *sarcolemma*). In a resting skeletal muscle cell, the interior of the cell is negatively charged relative to the extracellular fluid outside, sitting at a steady **resting membrane potential of approximately −70 mV to −90 mV**.

This negative voltage gradient is actively maintained by specialized transmembrane proteins called **Sodium-Potassium ATPase Pumps (Na^+/K^+ pumps)**. For every molecule of ATP consumed, this biological battery pump actively expels 3 sodium ions (Na^+) out of the cell while pulling only 2 potassium ions (K^+) into the cell. This unequal ionic transport, combined with resting membrane permeability to potassium, creates an electrochemical gradient: high Na^+ concentration outside, high K^+ concentration inside.

1. **Action Potential Arrival:** The action potential travels down the motor neuron axon to the neuromuscular junction, triggering the influx of calcium ions into the presynaptic terminal.

2. **Acetylcholine Release:** Synaptic vesicles fuse with the presynaptic membrane, releasing acetylcholine (ACh) neurotransmitter across the synaptic cleft into the synaptic gutter.

3. **Depolarization Wave Initiation:** ACh binds to nicotinic receptors on the motor endplate, opening ligand-gated ion channels. Sodium ions (Na+) rush catastrophically down their concentration gradient into the muscle cell, flipping the local transmembrane potential from −80 mV to +30 mV.

4. **Action Potential Propagation:** Voltage-gated sodium channels in adjacent regions of the sarcolemma detect this voltage shift and open sequentially, propagating a Muscle Fiber Action Potential (MFAP) outward in both directions toward the muscle tendons at 3 to 5 meters per second.

5. **Repolarization & Calcium Recovery:** Sodium channels inactivate and voltage-gated potassium channels (K+) open, allowing K+ to rush out of the cell to restore the −80 mV resting potential. Meanwhile, the depolarization wave travels down transverse tubules (T-tubules), triggering calcium release from the sarcoplasmic reticulum to initiate actin-myosin cross-bridge mechanical cycling.

The Goldman-Hodgkin-Katz Equation: Modeling Transmembrane Resting Potential

$$V_{membrane}(t) = \frac{RT}{F} \ln \left(\frac{P_{K}[K^{+}]_{out} + P_{Na}[Na^{+}]_{out} + P_{Cl}[Cl^{-}]_{in}}{P_{K}[K^{+}]_{in} + P_{Na}[Na^{+}]_{in} + P_{Cl}[Cl^{-}]_{out}} \right)$$

As the depolarization and repolarization waves travel along thousands of muscle fibers, they generate tiny cylindrical electrical currents in the surrounding biological tissue. Because living tissue (interstitial fluid, fat, skin) is an electrolytic volume conductor containing dissolved salt ions, these micro-currents establish an electrical potential field that propagates all the way to the surface of the skin.

The sum of all individual Muscle Fiber Action Potentials (MFAPs) originating from a single motor unit is called a **Motor Unit Action Potential (MUAP)**. When a motor unit fires repetitively, it generates a **Motor Unit Action Potential Train (MUAPT)**. The electromyographic signal measured at the skin is the grand linear spatial and temporal superposition of hundreds of overlapping MUAPTs:

Mathematical Model of Surface EMG: Linear Superposition of M Active Motor Units

$$s_{EMG}(t) = \sum_{i=1}^{M} \sum_{k=-\infty}^{\infty} h_i(t - \tau_{i,k}) + n(t)$$

Where $h_i(t)$ represents the spatial wave shape of the i-th motor unit action potential arriving at the skin, $\tau_{i,k}$ is the timing of the k-th firing of that motor unit, and $n(t)$ represents additive background physiological and instrumentation noise.

> 📖 **B.P. Lathi Textbook Reference (Section 1.4 & 1.5): Linear Superposition of Physical Sources**
>
> In Chapter 1 of *Linear Systems and Signals*, B.P. Lathi establishes the principle of **superposition** for linear systems: if multiple independent excitation sources stimulate a linear medium simultaneously, the aggregate response is simply the algebraic sum of the individual responses. In surface electromyography, the human limb behaves as a linear conductive volume medium. Each contracting motor unit acts as an elementary current dipole source. The voltage captured by surface electrodes is precisely the continuous linear superposition of these hundreds of asynchronous biological dipole waves!

1.3 Surface EMG (sEMG) vs Intramuscular Fine-Wire EMG

In biomedical engineering and neurophysiology, there are two distinct methodologies for capturing electromyographic signals: **Surface Electromyography (sEMG)** and **Intramuscular (Needle / Fine-Wire) Electromyography**. Understanding their fundamental trade-offs explains why sEMG is mandatory for interactive consumer applications like VR weightlifting.

Feature / Dimension	Surface EMG (sEMG) [Our Project]	Intramuscular Needle / Fine-Wire EMG
Invasiveness	Completely non-invasive; adhesive surface electrodes placed on skin	Invasive; hypodermic needle or hooked wire inserted directly into muscle
Spatial Resolution	Global muscle view; captures aggregate activity of hundreds of motor units	Hyper-local; captures isolated action potentials of 1 to 5 individual muscle fibers
Safety & Comfort	Zero pain, zero infection risk, completely safe for interactive games	Painful, risk of hematoma or infection, requires clinical medical oversight
Dynamic Movement	Allows unrestricted limb movement, bicep curls, athletic motions	Limb movement causes needle displacement, severe pain, and tissue tearing
Signal Amplitude	50 µV to 5,000 µV (attenuated by subcutaneous fat and skin layer)	500 µV to 20,000 µV (electrodes touch active muscle membranes directly)
High-Frequency Bandwidth	20 Hz to 450 Hz (tissue acts as a biological lowpass filter)	10 Hz to 2,000+ Hz (captures rapid needle-sharp spikes)
Primary Use Cases	VR/AR gaming, prosthetics control, sports biomechanics, ergonomics	Neuromuscular pathology diagnosis (ALS, myasthenia gravis, neuropathies)

For our VR weightlifting system, surface EMG is the only practical solution. While skin and subcutaneous fat act as a biological spatial low-pass filter (smoothing out individual sharp action potentials), this spatial averaging is actually an advantage for cybernetics: instead of tracking whether single motor unit #42 fired at millisecond 105, our VR game wants to know the **aggregate gross contractile force** generated by the entire muscle belly!

1.4 Biomechanics of the Biceps Brachii: The Ideal Cybernetic Transducer

The human arm contains several muscles responsible for moving the elbow joint, including the *biceps brachii*, *brachialis*, and *brachioradialis*. For SSRP Project 7, the **biceps brachii** was selected as our primary biological control actuator. Why is the biceps so uniquely suited for this cybernetic experiment?

Superficial Anatomical Location — Anatomy

The biceps brachii sits directly beneath the skin on the anterior compartment of the upper arm, separated from the surface by only a thin layer of subcutaneous fascia, minimizing signal loss.

High Signal-to-Noise Ratio (SNR) — Signal Quality

Because of its large cross-sectional area and dense motor unit population, the biceps produces robust electrical signals up to 3,000–5,000 µV during maximal curls, easily towering above noise.

Clear Biomechanical Action — Kinematics

The biceps has two distinct heads (long head and short head) originating from the scapula and inserting on the radial tuberosity. It is the primary flexor of the elbow and supinator of the forearm.

Intuitive User Mental Model — Psychology

Every human intuitively understands how to flex their biceps. In a weightlifting simulation, asking a player to flex their arm to lift a dumbbell creates an instantaneous neurological mapping in the motor cortex.

The biceps brachii functions as a third-class lever in classical Newtonian mechanics. The fulcrum is the elbow joint, the effort force is applied by the biceps tendon inserting into the radius roughly 3 to 5 cm distal to the joint axis, and the load force is applied at the hand (holding a 6 kg dumbbell ~35 cm away). Because the effort arm is much shorter than the load arm, the mechanical advantage is less than 1.0 (typically around 0.1).

To hold a 6 kg dumbbell stationary in the hand, the biceps tendon must generate over **400 to 600 Newtons of tensile force**! This massive mechanical tension requires recruiting thousands of motor units firing simultaneously, creating a roaring electrical storm beneath our surface electrodes.

1.5 Voltage Amplitudes and Frequency Content

To design appropriate digital filters and analog front-end circuits, we must quantitatively characterize the amplitude and frequency properties of the raw electromyographic signal.

Resting State Baseline

5 µV – 30 µV

When the arm is fully supported and relaxed, electrical activity is minimal, consisting of baseline thermal noise and occasional spontaneous miniature endplate potentials.

Moderate Contraction (Unloaded Curl)

100 µV – 800 µV

Lifting the empty forearm against gravity recruits fatigue-resistant Type I motor units firing at 10 to 20 Hz, generating clear rhythmic bursts.

Maximal Voluntary Contraction (6 kg Load)

1,500 µV – 5,000 µV

Curling a heavy weight against gravity recruits large Type II motor units. Hundreds of MUAP trains overlap asynchronously, creating peak-to-peak voltages up to 5 mV.

Dominant Spectral Energy Band

20 Hz – 500 Hz

Over 95% of total sEMG power is concentrated between 20 Hz and 450 Hz, with maximum energy density peaking between 60 Hz and 130 Hz.

⚠️ Critical Pitfall & Warning: The Physiological Muscle Fatigue Phenomenon

When an athlete holds a heavy dumbbell for an extended duration, muscle fatigue sets in. Biochemically, lactic acid accumulates, intracellular pH drops, and muscle fiber conduction velocity slows from 4 m/s down to 2.5 m/s.

In the frequency domain, this causes the **Median Frequency (MDF)** and **Mean Power Frequency (MPF)** of the EMG spectrum to progressively shift toward lower frequencies (e.g., from 110 Hz down to 75 Hz). In Chapter 10, we will examine how our calibration algorithm accommodates these physiological shifts so the virtual dumbbell doesn't drop prematurely as the user gets tired!

PART I: THE HUMAN BODY AS AN ELECTRICAL MACHINE • CHAPTER 2

Data Collection Hardware — The BioRadio Acquisition System

Deconstructing the biomedical instrumentation chain: from the electrochemical electrode-skin interface and differential instrumentation amplifiers to delta-sigma analog-to-digital conversion and Bluetooth telemetry.

2.1 The Electrode-Skin Interface: Bridging Chemistry and Electronics

The biggest hurdle in biosignal engineering is that the human body does not use copper wires and valence electrons to transmit electrical signals; it uses dissolved chemical salt ions (Na^+, K^+, Cl^-, Ca^{2+}) diffusing through water. An electronic instrument like the Great Lakes NeuroTechnologies (GLNT) BioRadio, on the other hand, operates exclusively with electron currents flowing through silicon and copper traces.

The device that performs this bidirectional translation between ionic currents in human tissue and electronic currents in a circuit is the **electrode**. Specifically, for surface electromyography, we employ **Silver/Silver Chloride (Ag/AgCl) non-polarizable surface electrodes** coated with a specialized conductive electrolyte gel.

> 🔬 **Technical Fact & Definition: The Half-Cell Chemical Battery**
>
> When a silver metal plate coated with silver chloride touches an electrolytic gel containing chloride ions (Cl^-), a reversible electrochemical oxidation-reduction reaction occurs:
>
> $$Ag + Cl^- \leftrightarrow AgCl + e^-$$
>
> When current flows from the tissue into the electrode, silver atoms oxidize into AgCl, releasing electrons into the lead wire. When current flows the other direction, AgCl reduces back into silver, absorbing electrons. This reversible reaction establishes a stable, low-noise half-cell potential (≈ +222 mV vs standard hydrogen electrode) that does not polarize or build up capacitive charge under microvolt biosignals!

The Skin Barrier and Electrical Equivalent Circuit

Human skin is an exceptional electrical insulator designed by evolution to protect us from environmental hazards. The outermost layer, the *stratum corneum*, consists of flattened, dead, keratinized epithelial cells with virtually no water content. An unprepared dry skin surface can present an electrical impedance exceeding **100,000 Ω (100 kΩ) to 1 MΩ**!

If electrode impedance is too high, two catastrophic problems arise:

- **Signal Attenuation:** High source impedance forms a voltage divider with the amplifier's input impedance, dramatically weakening the microvolt EMG signal.
- **Noise Susceptibility:** High impedance unbalances the differential inputs, causing external 50 Hz power-line electromagnetic hum to couple directly into the measurement channel.

1. **Skin Abrasion (Dead Cell Removal):** Gently prep the skin over the biceps muscle belly using a mild abrasive paste or alcohol prep pad to scrub away dead stratum corneum cells and sebum oils.
2. **Conductive Electrolyte Gel Application:** Apply conductive chloride gel. The gel penetrates remaining microscopic skin crevices, establishing an aqueous ionic bridge with deeper viable epidermis.
3. **Electrode Adhesion & Stabilization:** Firmly press Ag/AgCl pre-gelled adhesive snap electrodes onto the skin, ensuring zero air bubbles. Air gaps act as microscopic capacitors that induce motion noise when the arm flexes.
4. **Impedance Check:** Verify that skin-to-electrode contact impedance drops below **5 kΩ to 10 kΩ**, ensuring maximum signal fidelity and common-mode rejection.

2.2 Differential Recording and Ground Reference

How do we measure a 500 µV muscle signal when the surrounding room is filled with 230-volt AC power lines radiating electromagnetic noise that induces several volts of static hum onto the human body? If we placed a single electrode on the biceps and measured its voltage relative to the earth ground, our signal would be 99.9% power-line buzzing and 0.1% muscle!

The biomedical solution is **Differential Bipolar Recording**. Instead of one electrode, we place two active electrodes along the biceps muscle belly:

- **Non-Inverting Input (IN+):** Placed over the proximal belly of the biceps brachii.
- **Inverting Input (IN−):** Placed approximately 20 mm distally along the longitudinal axis of the same muscle belly, parallel to muscle fiber orientation.
- **Ground Reference (GND):** Placed on an electrically neutral, bony landmark with zero underlying muscle tissue, such as the *olecranon process* of the elbow or the *acromion* of the shoulder.

> 💡 **Intuitive Analogy: Two Microphones in a Roaring Stadium**
> Imagine you want to record the whisper of two people having a conversation inside a roaring sports stadium where 50,000 fans are shouting at the top of their lungs.
>
> If you use one microphone, all you record is the deafening stadium roar.
>
> Instead, place **two identical microphones** right next to each other, just 2 inches apart, right in front of the whispering person. Both microphones pick up the identical roaring noise of 50,000 fans from far away (the *Common-Mode Signal*). But Microphone 1 is slightly closer to the speaker's mouth than Microphone 2 (the *Differential Signal*).
>
> Now subtract the two microphone signals: **Audio = Mic 1 − Mic 2**. The stadium roar subtracts to zero, leaving only the pristine whisper! That is exactly what differential EMG recording accomplishes!

Mathematical Formulation of Differential Signal Amplification

$$V_{out}(t) = A_d \cdot (V_{IN+} - V_{IN-}) + A_{cm} \cdot \left(\frac{V_{IN+} + V_{IN-}}{2} \right)$$

2.3 The Instrumentation Amplifier and CMRR

At the physical core of the BioRadio hardware sits a specialized, ultra-high-precision integrated circuit called an **Instrumentation Amplifier (In-Amp)**. Standard operational amplifiers (op-amps) are insufficient because they lack matched input impedances and have high input bias currents.

A classic instrumentation amplifier employs a **three-op-amp topology**. The first stage consists of two non-inverting buffer op-amps with a shared gain resistor (R_G). This provides an astronomical input impedance (exceeding **10^{10} Ω or 10 GΩ**), ensuring that the amplifier draws virtually zero current from the skin, preventing signal loading. The second stage is a differential subtractor op-amp configured with precision laser-trimmed resistors.

> 🔬 **Technical Fact & Definition: Common-Mode Rejection Ratio (CMRR)**
> The measure of an instrumentation amplifier's ability to reject identical noise on both inputs while amplifying the tiny differential difference is the **Common-Mode Rejection Ratio (CMRR)**, expressed in decibels (dB):
>
> $$\mathrm{CMRR\ (dB)} = 20 \cdot \log_{10} \left(|A_d / A_{cm}| \right)$$
>
> Where A_d is differential gain (typically 1,000× to 5,000×) and A_{cm} is common-mode gain (ideally zero).
>
> The BioRadio boasts a CMRR exceeding **110 dB** at 50/60 Hz! A CMRR of 100 dB means the amplifier suppresses common-mode power-line noise by a factor of **100,000 to 1** relative to the differential muscle signal!

2.4 Analog-to-Digital Conversion (ADC)

Once the biological signal is amplified and pre-filtered in the analog domain, it must be converted from a continuous analog voltage into a stream of discrete binary numbers that our Python script can process.

The BioRadio features a high-resolution **16-bit Delta-Sigma (ΔΣ) Analog-to-Digital Converter**. A 16-bit converter divides its full-scale input voltage range into 2^{16} = 65,536 discrete quantization levels. If the BioRadio's input dynamic range is set to ±10 mV (±10,000 µV), the quantization step size (q) is:

Quantization Step Resolution of the BioRadio 16-Bit ADC

$$q = \frac{V_{max} - V_{min}}{2^B} = \frac{10{,}000\,\mu\text{V} - (-10{,}000\,\mu\text{V})}{65{,}536} = \frac{20{,}000\,\mu\text{V}}{65{,}536} \approx 0.305\,\mu\text{V}$$

With a resolution of approximately **0.3 µV per digital step**, the system can effortlessly resolve even the subtlest 10 µV resting twitches without introducing digital rounding noise. The theoretical Signal-to-Quantization-Noise Ratio (SQNR) for an ideal B-bit

ADC is given by:

Theoretical Signal-to-Quantization-Noise Ratio for a 16-Bit ADC

$$\text{SQNR (dB)} = 6.02 \cdot B + 1.76 = 6.02 \times 16 + 1.76 \approx 98.08\,\text{dB}$$

2.5 The Nyquist-Shannon Sampling Theorem

When converting continuous-time physical reality into discrete-time computer data, how fast must we sample? If we sample too slowly, we commit the cardinal sin of digital signal processing: **Aliasing**.

> 📖 **B.P. Lathi Textbook Reference (Chapter 8: The Sampling Theorem): Nyquist Rate and Anti-Aliasing**
> In Chapter 8 of *Linear Systems and Signals*, B.P. Lathi establishes the landmark **Nyquist-Shannon Sampling Theorem**:
>
> *A continuous-time bandlimited signal x(t) with highest frequency component f_{max} can be uniquely and perfectly reconstructed from its discrete samples x[n] = x(n·T_s) if and only if the sampling frequency f_s satisfies:*
>
> $$f_s \geq 2 \cdot f_{max}$$
>
> The minimum allowable sampling frequency $2 \cdot f_{max}$ is known as the **Nyquist Rate**. If this condition is violated, high-frequency energy folds back across the Nyquist folding frequency ($f_s/2$) into the passband, creating irreversible ghost frequencies (aliases) that permanently corrupt the data!

Parameter	Theoretical Requirement	Project 7 Implementation	Engineering Rationale
Maximum EMG Frequency (f_{max})	450 Hz – 500 Hz	450 Hz Bandlimit	99% of physiological sEMG power resides below 450 Hz
Nyquist Minimum Rate ($2 \cdot f_{max}$)	900 Hz – 1,000 Hz	1,000.0 Hz Selected	Guarantees Nyquist compliance with comfortable safety margin
Hardware Sampling Rate (f_s)	≥ 1,000 Hz	`fs = 1000.0 Hz`	Exactly 1 sample every 1.0 millisecond (T_s = 0.001 s)
Analog Anti-Aliasing Filter	Lowpass cut < $f_s/2$	Analog 4th-order Butterworth	Hardware lowpass at 450 Hz inside BioRadio before ADC conversion

2.6 Wireless Bluetooth Transmission Protocol

The digitized samples are not transmitted down a physical tether that could trip the VR athlete or pull electrodes off their skin. The BioRadio encapsulates the digitized samples and broadcasts them wirelessly over standard **Bluetooth 2.1 + EDR (Enhanced Data Rate)** using the Serial Port Profile (SPP / RFCOMM).

At a sampling rate of 1,000 Hz with 16-bit samples across multiple bio-potential channels, the raw data rate is approximately 16,000 to 64,000 bits per second. The BioRadio's internal microcontroller buffers samples into small transmission frames dispatched every 10 to 20 milliseconds.

On the receiving Windows workstation, the vendor-provided dynamic link library `BioRadioSDK.dll` communicates with the Bluetooth virtual COM port, validates frame header bytes and CRC checksums, unpacks the 16-bit integers, applies factory calibration scale factors, and converts the raw digital words into engineering units of **microvolts (µV)** ready for our Python signal processing pipeline.

PART II: THE NOISE MONSTERS AND WHY FILTERING IS ESSENTIAL • CHAPTER 3

Sources of Noise and Contamination in Electromyographic Signals

Diagnosing the electromagnetic, mechanical, and electrophysiological noise contaminants that corrupt surface EMG, and exposing the fundamental 'zero-sum trap' of raw bipolar recordings.

3.1 The Power Line Interference Monster (50 Hz / 60 Hz Mains Hum)

Of all the noise sources plaguing biomedical engineers, **power line interference** is the most pervasive, energetic, and aggressive. In India, Europe, and most of Asia, AC electrical grids oscillate at **50 Hz** (while North America uses 60 Hz). Every wall outlet, extension cord, fluorescent tube light, desktop computer, and VR base station radiates an oscillating 50 Hz electromagnetic field.

Because the human body is predominantly water filled with conductive electrolytes, the human body acts as a literal **receiving antenna**! This electromagnetic coupling manifests in two distinct physics pathways:

- **Capacitive Coupling:** Displacement currents flow through stray capacitance between overhead 230V mains wiring and the surface of the user's skin ($C_{line\text{-}body} \approx 5$ to 10 pF). This dumps a common-mode AC current through the body into earth ground, creating voltage oscillations of several volts on the skin.
- **Inductive Coupling:** Magnetic fields created by high-current appliances (such as the computer's power supply unit or VR tracking hardware) induce eddy currents and voltages in the loop formed by the electrode lead cables ($V_{ind} = -d\Phi_B / dt$).

> 🔬 **Technical Fact & Definition: The 50 Hz EMG Tragedy**
>
> In audio engineering or speech processing, a 50 Hz hum can simply be removed by applying a steep high-pass filter that cuts off everything below 80 Hz, because human speech has very little information at 50 Hz.
>
> In surface electromyography, however, **50 Hz sits dead center in the most energetic, biologically vital band of the EMG spectrum** (which spans 20 Hz to 150 Hz)! If you casually apply a wide filter that deletes everything around 50 Hz, you destroy over 30% of the muscle's true physiological energy! We require a surgically precise, knife-thin notch filter to cut out 50.0 Hz without disturbing 48 Hz or 52 Hz.

3.2 Motion Artifacts and Cable Whipping (< 20 Hz)

The second major noise category is **motion artifacts**. When an athlete performs a dynamic bicep curl in VR, the arm flexes, the skin stretches, the muscle belly swells, and the electrode lead wires whip through the air.

Motion artifacts originate from two distinct physical mechanisms:

- **Electrode-Skin Gel Shearing:** When the muscle swells beneath the adhesive patch, mechanical shear forces deform the microscopic double layer of ionic charge between the skin and the Ag/AgCl conductive gel. This deformation perturbs the half-cell potential, generating massive, low-frequency voltage swings of **several millivolts (1,000 to 5,000 µV)**!
- **Triboelectric Cable Whipping:** As lead cables move, frictional contact between the internal copper conductor and the outer plastic insulation generates static charges (the triboelectric effect), creating spurious low-frequency voltage spikes.

> 💡 **Intuitive Analogy: Weighing a Hamster on an Earthquake Fault**
>
> Imagine you want to weigh a tiny 30-gram hamster standing on a sensitive digital kitchen scale. But someone is violently shaking the kitchen table up and down like an earthquake!
>
> The reading on the scale will swing wildly between −500 grams and +1,500 grams. The wild swinging is not the weight of the hamster; it is the physical shaking of the table. In surface EMG, the muscle's action potentials are the hamster, and the arm's movement shaking the electrode is the earthquake. Fortunately, the earthquake happens slowly (below 20 Hz), while the hamster runs fast (above 20 Hz)!

Because human limbs and heavy dumbbells cannot physically oscillate faster than 10 to 15 times per second due to mechanical inertia, **motion artifacts are strictly bandlimited to frequencies below 20 Hz**. This physical principle allows us to eradicate motion artifacts using a dedicated high-pass filter cutoff at 20.0 Hz.

3.3 High-Frequency Thermal and Radio Frequency Noise (> 450 Hz)

At the upper end of the frequency spectrum, signals above 450 Hz contain virtually zero physiological muscle activity. Living muscle membranes have a finite capacitive charging time and a finite action potential conduction velocity that prevents them from generating significant electrical energy beyond 400 to 450 Hz.

Any electrical fluctuations recorded above 450 Hz represent unwanted instrumentation and environmental noise:

- **Johnson-Nyquist Thermal Noise:** Inherent thermodynamic agitation of charge carriers inside the resistive electrode-skin interface and amplifier resistors ($v_n = \sqrt{(4\ k_B\ T\ R\ \Delta f)}$). This generates a flat, Gaussian white-noise floor across all frequencies.
- **Radio-Frequency (RF) Interference:** Electromagnetic emissions from Wi-Fi routers (2.4 GHz and 5 GHz), Bluetooth transmitters, cellular antennas, and high-frequency switching DC-DC power converters inside the VR headset and PC.
- **ADC Clock Jitter:** High-frequency clock noise and sampling jitter within the analog front-end.

A digital low-pass cutoff at 450.0 Hz suppresses these high-frequency hiss contaminants, purifying the signal and ensuring robust Nyquist headroom.

3.4 Muscle Crosstalk: Volume Conduction from Neighboring Muscles

When measuring the biceps brachii, the electrodes do not exist in an isolated vacuum. The upper arm also contains the *brachialis* (situated deep to the biceps), the *coracobrachialis* (medial compartment), and the powerful *triceps brachii* (posterior compartment).

When the user stabilizes their elbow or grips a heavy weight, neighboring muscles contract. Their electrical currents propagate through the conductive interstitial fluid and subcutaneous fat—a physical phenomenon known as **volume conduction**. The resulting voltage arriving at the biceps electrodes is called **muscle crosstalk**.

Biological tissue acts as a spatial low-pass filter: high frequencies attenuate rapidly over distance, so crosstalk signals typically appear as low-amplitude, low-frequency blurred waveforms. By following standardized SENIAM (*Surface ElectroMyoGraphy for the Non-Invasive Assessment of Muscles*) guidelines—placing electrodes firmly on the center of the biceps muscle belly at an inter-electrode distance (IED) of exactly 20 mm—we maximize the local biceps signal while minimizing crosstalk.

3.5 Baseline Wander and DC Electrode Offset

If you plug raw electrodes directly into a computer and plot the unadorned voltage, you will immediately notice that the signal does not center around 0.0 µV. Instead, it rides upon a massive, slowly drifting DC offset of **±10,000 µV to ±50,000 µV (±10 to ±50 mV)**!

This huge DC offset is caused by electrochemical half-cell potential imbalances between the two electrodes, differences in local skin perspiration, skin temperature gradients, and slow chemical equilibrium drift. If left uncorrected, this DC offset would completely destroy digital RMS envelope calculations, as squaring a 30,000 µV DC offset produces an astronomical false muscle activation value!

3.6 The "Zero-Sum Trap": Why Raw EMG Averages to Zero!

Now we arrive at the most vital conceptual breakthrough in all of biosignal processing—a counterintuitive fact that trips up almost every beginner in biomedical computing:

> ⚠️ **Critical Pitfall & Warning: The Great Zero-Sum Trap**
>
> **A RAW EMG SIGNAL HAS AN AVERAGE VALUE OF ZERO, NO MATTER HOW HARD YOU ARE FLEXING!**
>
> If you ask an athlete to curl an enormous 20 kg barbell until their biceps are trembling with maximal exertion, and you take the mathematical average (mean) of all the raw data points recorded over that 5-second window, the result is:
>
> $$\text{Mean}(\text{Raw EMG}) \approx 0.0\ \mu\text{V}$$
>
> If a naive game programmer tries to map the raw EMG signal directly to the virtual arm in Unity, the virtual arm will never move an inch! Why does this happen?

The Electrophysiological Proof: Why MUAPs Are Biphasic Waves

Consider what happens physically as an action potential travels down a muscle fiber underneath our two differential electrodes (IN+ and IN−):

A **Approaching the First Electrode (IN+):** As the negative depolarization wave approaches IN+, it creates an external electrical dipole. The amplifier measures a positive voltage deflection: V = (V_IN+ − V_IN−) > 0.

B **Dead Center Between Electrodes:** When the depolarization wave sits exactly halfway between IN+ and IN−, both electrodes experience identical electrical potentials. The differential amplifier subtracts them: V = (V_IN+ − V_IN−) = 0.0 µV.

C **Passing the Second Electrode (IN−):** As the wave continues traveling distal and passes beneath IN−, IN− becomes intensely negative while IN+ has returned to baseline. The amplifier measures a negative voltage deflection: V = (V_IN+ − V_IN−) < 0.

D **Perfect Symmetry Cancellation:** Because the positive electrical lobe is identical in area and charge to the negative electrical lobe, the mathematical integral of this wave across time is exactly zero: ∫ v(t) dt = 0.0!

Mathematical Proof of the Zero-Mean Property of Bipolar EMG

$$\mu = \lim_{T \to \infty} \frac{1}{T} \int_{0}^{T} s_{EMG}(t)\,dt = 0.0\,\mu\text{V}$$

Because every single motor unit action potential is a biphasic or triphasic electrical dipole that swings equally above and below zero, summing hundreds of them together still yields an alternating signal whose average is zero.

What changes when you flex harder is NOT the average voltage; it is the **variance, amplitude envelope, and total electrical power** of the signal! In Chapter 9, we will see how rectification and Root Mean Square (RMS) envelope detection solve the zero-sum trap by converting bidirectional oscillations into a unipolar physical power curve.

Noise Contaminant	Frequency Range	Physical Origin	Filter Weapon Employed
DC Offset & Baseline Wander	0 Hz (Pure DC)	Electrode half-cell potential drift & sweat	High-Pass Bandpass Cutoff (20.0 Hz)
Motion Artifacts	< 20 Hz	Skin stretching, gel shear, cable movement	High-Pass Bandpass Cutoff (20.0 Hz)
50 Hz Mains Hum	50.0 Hz ± 1 Hz	Capacitive/magnetic coupling from AC power grid	50 Hz IIR Notch Filter (Q=30)
High-Frequency Thermal Noise	> 450 Hz	Johnson noise in skin/resistors, RF, ADC jitter	Low-Pass Bandpass Cutoff (450.0 Hz)
Zero-Sum Alternation	All Frequencies	Biphasic dipole nature of propagating action potentials	Full-Wave Rectification + Moving Window RMS

PART II: THE NOISE MONSTERS AND WHY FILTERING IS ESSENTIAL • CHAPTER 4

Fundamentals of Discrete-Time Digital Signal Processing (DSP)

Establishing the core mathematical foundations of linear time-invariant systems, the z-transform, frequency response, filter specifications, and the mechanics of real-time streaming state preservation.

4.1 What is a Digital Filter? Linear Time-Invariant (LTI) Systems

In the continuous analog domain, a filter is a physical circuit assembled from capacitors, inductors, and resistors that attenuates certain frequencies while allowing others to pass. In the discrete digital domain, a **digital filter is an algorithm—a mathematical recipe** that takes an input sequence of numbers x[n] and computes a transformed output sequence of numbers y[n].

The digital filters we design for biomedical applications belong to the mathematical class of **Linear Time-Invariant (LTI) Systems**:

- **Linearity (Superposition):** If an input $x_1[n]$ produces output $y_1[n]$ and input $x_2[n]$ produces output $y_2[n]$, then an input of (α $x_1[n]$ + β $x_2[n]$) produces an output of (α $y_1[n]$ + β $y_2[n]$). The filter does not distort the proportional scaling or additive combination of signals.
- **Time-Invariance:** Shifting the input sequence in time by k samples simply shifts the output sequence by the exact same k samples: if x[n] → y[n], then x[n-k] → y[n-k]. The system's rules do not spontaneously change at 3:00 PM vs 3:05 PM.

An LTI system is completely characterized by its **unit impulse response**, denoted h[n]—the output sequence produced when the input is a single discrete unit Kronecker impulse δ[n] (where δ[0] = 1 and δ[n] = 0 for all n ≠ 0). For any arbitrary input x[n], the output is obtained via the **discrete-time convolution sum**:

The Discrete-Time Convolution Sum: Heart of LTI Filter Theory

$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n-k]$$

💡 **Intuitive Analogy: The Magical Coffee Sieve**

Imagine pouring a bucket of mixed materials through a series of wire mesh sieves. The bucket contains large rocks (motion artifacts < 20 Hz), fine coffee grounds (muscle signals 20–450 Hz), and microscopic airborne dust (thermal noise > 450 Hz).

A **Bandpass Filter** is a double sieve! The top sieve has large holes: it stops the big rocks (< 20 Hz) while letting everything else fall through. The bottom sieve has ultra-fine mesh: it catches the valuable coffee grounds (20–450 Hz) while allowing the dust to blow away (> 450 Hz).

A **Notch Filter** is like a tiny pair of robotic tweezers that plucks out a single intrusive fly sitting at 50 Hz, leaving all the coffee grounds undisturbed!

4.2 The z-Transform and the Transfer Function H(z)

While time-domain convolution allows us to calculate filter outputs, it is computationally cumbersome for analyzing frequency behavior. To gain deep analytical power, we transform discrete signals into the complex frequency domain using the **z-Transform**.

The two-sided z-transform of a discrete-time signal x[n] is defined as:

Definition of the Discrete-Time z-Transform

$$X(z) = \mathcal{Z}\{x[n]\} = \sum_{n=-\infty}^{\infty} x[n] \cdot z^{-n}$$

Where z is a continuous complex variable that can be expressed in polar form as $z = r e^{j\Omega}$, with magnitude r and digital angular frequency $\Omega = 2\pi f / f_s$ (measured in radians per sample).

The ratio of the z-transformed output Y(z) to the z-transformed input X(z) defines the **System Transfer Function**, H(z):

Rational Transfer Function Form: Numerator Zeros and Denominator Poles

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\sum_{k=0}^{M} b_k z^{-k}}{1 + \sum_{k=1}^{N} a_k z^{-k}} = b_0 \cdot \frac{\prod_{i=1}^{M} (1 - c_i z^{-1})}{\prod_{k=1}^{N} (1 - d_k z^{-1})}$$

The roots of the numerator polynomial (c_i) are the **zeros** of the system (frequencies where the filter output drops to zero). The roots of the denominator polynomial (d_k) are the **poles** of the system (frequencies where the system's natural resonance modes

reside).

The Unit Circle and Frequency Response

When the complex variable z evaluates along the **unit circle** (where radius r = 1, so $z = e^{j\Omega}$), the z-transform simplifies to the **Discrete-Time Fourier Transform (DTFT)**. The system's frequency response is obtained by evaluating H(z) directly on the unit circle:

Decomposition of Frequency Response into Magnitude and Phase

$$H(e^{j\Omega}) = |H(e^{j\Omega})| \cdot e^{j\angle H(e^{j\Omega})}$$

- **Magnitude Response $|H(e^{j\Omega})|$:** Determines how much each frequency is amplified or attenuated (gain).
- **Phase Response $\angle H(e^{j\Omega})$:** Determines how much each frequency is delayed or phase-shifted in time.

4.3 Practical Filter Specifications

In the real world, mathematics does not allow us to build an "ideal brick-wall filter" (a rectangular filter with infinitely steep vertical drop-offs) because such a filter requires an infinite impulse response extending backward into the future, violating physical causality.

> 📖 **B.P. Lathi Textbook Reference (Section 4.10-5): Practical Filter Design Boundaries**
> In Section 4.10-5 of *Linear Systems and Signals*, B.P. Lathi formalizes the standard engineering tolerances for physical filter design:
>
> 1. **Passband ($0 \le |\omega| \le \omega_p$):** The band of frequencies transmitted with minimal attenuation, bounded by allowable ripple ($1 - \delta_p \le |H| \le 1 + \delta_p$).
> 2. **Stopband ($\omega \ge \omega_s$):** The band of unwanted noise frequencies attenuated below a minimum threshold (δ_s, often −40 dB or −60 dB).
> 3. **Transition Band ($\omega_p < \omega < \omega_s$):** The buffer zone over which the filter smoothly transitions from passband to stopband.
> 4. **Cutoff Frequency (ω_c):** The half-power point where filter power drops by 50%, corresponding to a **−3 dB** voltage drop ($|H(j\omega_c)| = 1/\sqrt{2} \approx 0.707$).

The Universal Definition of the −3 dB Filter Cutoff Frequency

$$\text{Gain (dB)} = 20 \cdot \log_{10} |H(e^{j\Omega})| \implies \text{At Cutoff: } 20 \log_{10}(0.707) = -3.01\,\text{dB}$$

4.4 The Bandpass Concept for sEMG: 20 Hz to 450 Hz

To satisfy both electrophysiology and discrete-time sampling requirements, our SSRP Project 7 pipeline establishes a dedicated **Bandpass Filter** spanning from f_{low} = 20.0 Hz to f_{high} = 450.0 Hz.

Lower Cutoff: 20.0 Hz — High-Pass Action
Eradicates motion artifacts caused by skin stretching, cable whipping, and electrochemical DC baseline drift, while preserving over 98% of physiological MUAP energy.

Upper Cutoff: 450.0 Hz — Low-Pass Action
Suppresses high-frequency thermal noise, RF pickup, and switching power supply emissions. Protects the Nyquist boundary at $f_s/2$ = 500 Hz.

Dynamic Headroom Safeguard — Adaptive Code Logic
In `emg_methods.py`, the upper cutoff is dynamically calculated as: `high = min(450.0, 0.45 * self.fs)`, ensuring safe operation even if sampling rate changes.

Phase & Group Delay Balance — Cybernetics
Configured to ensure smooth temporal response so that muscle contractions register within 40 ms, well below human tactile reaction latency.

4.5 The Notch Filter Concept: 50 Hz Rejection

A bandpass filter alone cannot eliminate 50 Hz power-line interference, because 50 Hz sits comfortably inside the 20–450 Hz passband. We must cascade a specialized **Notch (Band-Stop) Filter** immediately following the bandpass stage.

A digital notch filter works by placing a pair of **complex-conjugate zeros directly on the unit circle** at the digital frequency corresponding to 50 Hz:

Location of Notch Zeros on the z-Plane Unit Circle

$$\Omega_0 = 2\pi \cdot \frac{f_0}{f_s} = 2\pi \cdot \frac{50.0}{1000.0} = 0.1\pi\,\text{rad/sample} \implies z_{zero} = e^{\pm j\,0.1\pi}$$

To prevent the notch from attenuating frequencies nearby (like 45 Hz or 55 Hz), a pair of **complex-conjugate poles** is placed directly behind the zeros at the same angle, but just inside the unit circle at radius r ≈ 0.95.

The sharpness of the notch is parameterized by its **Quality Factor (Q)**:

Quality Factor and −3 dB Bandwidth of the 50 Hz Notch Filter

$$Q = \frac{f_0}{\Delta f_{-3\text{dB}}} = \frac{50.0\,\text{Hz}}{30.0} \implies \Delta f_{-3\text{dB}} = \frac{50.0}{30.0} \approx 1.67\,\text{Hz}$$

With Q = 30.0, the notch has a razor-thin bandwidth of just 1.67 Hz. It completely obliterates 50.0 Hz interference while leaving 48 Hz and 52 Hz muscle potentials untouched!

4.6 Causality, Real-Time Streaming, and Initial Conditions

In academic signal processing textbooks, authors frequently demonstrate filtering using functions like `scipy.signal.filtfilt`. The `filtfilt` algorithm performs "forward-backward filtering": it filters the data forward, reverses the array in memory, and filters it backward. This cancels all phase lag, producing zero phase distortion.

> ⚠️ **Critical Pitfall & Warning: The Fatal Real-Time Flaw of filtfilt**
>
> `scipy.signal.filtfilt` **CANNOT BE USED IN A REAL-TIME VR GAME!**
>
> Forward-backward filtering requires you to have the ENTIRE signal recorded into the future before you can start filtering backward! In our VR weightlifting game, the future has not happened yet! You cannot filter backward from a dumbbell curl that the user will perform 3 seconds from now.
>
> A real-time game demands **Strictly Causal Filters**: the output y[n] at millisecond n can depend ONLY on present and past inputs (x[n], x[n-1], ...) and past outputs (y[n-1], ...).

The Challenge of Filter States across Streaming Chunks

In our architecture, the BioRadio delivers data in small streaming packets every 20 milliseconds (batches of 20 samples at f_s = 1000 Hz).

If you treat each 20-sample chunk as a brand-new, isolated array and pass it to a digital filter with zero initial conditions, the filter's internal delay registers start at zero. This creates a violent **transient glitch or pop** at the beginning of every single 20 ms chunk! Over a 1-second period, you will hear 50 synthetic popping clicks that utterly corrupt the RMS envelope.

To achieve flawless streaming continuity, the filter must **remember its internal state (z_i)** from the end of chunk k and pass it as the initial state condition into chunk k+1. In the next chapter, we inspect how Raima Ghosh implemented this state persistence in Python.

PART III: THE FOUR FILTER SUPERHEROES — DEEP TECHNICAL ANALYSIS • CHAPTER 5

Infinite Impulse Response (IIR) Filter — Raima's Method

An exhaustive investigation of recursive feedback architectures, Butterworth maximally flat pole placement, Second-Order Sections (SOS) numerical stabilization, and state persistence.

5.1 Recursive Feedback and Infinite Impulse Response (IIR)

The first filtering methodology in our comparative research was architected and implemented by student researcher **Raima Ghosh**. Raima's weapon of choice is the **Infinite Impulse Response (IIR) Digital Filter**.

What defines an IIR filter? In a conventional non-recursive calculation, the current output y[n] depends solely on current and past input samples (x[n], x[n-1], ...). In an IIR filter, however, the architecture introduces **recursive mathematical feedback**: the current output y[n] is computed from a linear combination of past input samples AND past output samples:

The General Linear Constant-Coefficient Difference Equation for IIR Filters

$$y[n] = \sum_{k=0}^{M} b_k \cdot x[n-k] - \sum_{k=1}^{N} a_k \cdot y[n-k]$$

Where b_k are the *feedforward coefficients*, a_k are the *feedback coefficients*, M is the feedforward order, and N is the feedback order.

> 💡 **Intuitive Analogy: The Grand Canyon Echo Chamber**
>
> Imagine standing on the rim of the Grand Canyon and shouting a single loud word: "HELLO!" (a discrete unit impulse δ[n]).
>
> The sound bounces off the far canyon wall and returns as an echo. But that echo bounces off the wall behind you and travels back across the canyon again, creating a second, slightly quieter echo! That second echo bounces again, producing a third, then a fourth, fading off into eternity.
>
> Because past outputs feed back into the canyon, the sound reverberates forever, decaying exponentially toward zero. That is why this architecture is called an **Infinite Impulse Response**: feed it one single pulse, and its internal mathematical feedback generates an infinite sequence of decaying numbers!

Why Choose an IIR Architecture? Extreme Efficiency

The fundamental engineering virtue of an IIR filter is its **unparalleled computational efficiency**. Because past outputs do the heavy lifting through feedback resonance, an IIR filter can achieve razor-sharp transition bands and aggressive noise suppression with an order of just 4 to 8. To achieve an identical transition sharpness using a non-recursive FIR filter, you would need over 100 to 200 taps!

Fewer mathematical operations per sample translates to microscopic CPU overhead, allowing our Python engine to run smoothly on low-power VR laptop backpacks.

5.2 Butterworth Filter Design: Maximally Flat Passband

There are several classical analog prototype filter families available to an engineer (Butterworth, Chebyshev Type I, Chebyshev Type II, and Elliptic). For biomedical electromyography, Raima specifically selected the **Butterworth Filter**. Why?

Chebyshev and Elliptic filters achieve steeper cutoff slopes, but they pay a terrible biological price: they introduce **equiripple oscillations (ripples)** into the passband. If an EMG filter has ±1.5 dB ripple in its passband, an 80 Hz action potential will be artificially amplified by 120%, while a 110 Hz action potential will be squashed to 85%! This artificial distortion of physiological muscle power cannot be tolerated.

📖 **B.P. Lathi Textbook Reference (Section 4.12-2): Butterworth Maximally Flat Filter Theory**

In Section 4.12-2 of *Linear Systems and Signals*, B.P. Lathi establishes the mathematical proof of the Butterworth filter's unique virtue: **Maximal Flatness**.

The magnitude squared response of an N-th order low-pass Butterworth filter is defined as:

$$|H(j\omega)|^2 = 1 / [1 + (\omega / \omega_c)^{2N}]$$

At ω = 0, the first **2N − 1 derivatives** of $|H(j\omega)|^2$ with respect to ω are identically equal to zero!

This mathematical property ensures the smoothest possible passband response without a single microscopic ripple, preserving the authentic physical power spectrum of the biceps brachii.

Butterworth Magnitude Equation and Half-Power Attenuation

$$|H(j\omega)| = \frac{1}{\sqrt{1 + \left(\frac{\omega}{\omega_c} \right)^{2N}}} \implies \text{At } \omega = \omega_c: |H(j\omega_c)| = \frac{1}{\sqrt{1 + 1}} = \frac{1}{\sqrt{2}} \approx 0.7071\,(-3\,\text{dB})$$

Order Selection: Why Raima Chose 4th Order (N = 4)

In our Python implementation, Raima invoked `signal.butter(4, (20.0, high), btype='bandpass', ...)`. Note that in SciPy, specifying order 4 for a bandpass filter constructs a **4th-order prototype that expands into an 8th-order bandpass system (8 complex poles)**.

The asymptotic attenuation slope of a Butterworth filter in the stopband is given by:

Asymptotic High-Frequency Rejection Rate of Raima's Filter

$$\text{Attenuation Slope} = -20 \cdot N\,\text{dB/decade} = -20 \times 4 = -80\,\text{dB/decade} \; (-24\,\text{dB/octave})$$

A rolloff of −80 dB per decade means that for every 10× increase in frequency outside the passband, noise is suppressed by a factor of **10,000 to 1**! This provides total eradication of low-frequency motion artifacts below 20 Hz without requiring an excessively high order that could introduce phase instability.

5.3 Second-Order Sections (SOS) Realization: Taming the Numerical Monster

Why does Raima's code specifically declare `output='sos'` instead of standard transfer function numerator and denominator polynomials `(b, a)`? This subtle choice represents a masterstroke of modern numerical engineering.

📖 **B.P. Lathi Textbook Reference (Section 4.12-3): Second-Order Sections (SOS) and Numerical Stability**

In Section 4.12-3 of *Linear Systems and Signals*, Lathi analyzes the catastrophic numerical instability that occurs when implementing high-order digital filters in direct polynomial form.

When an 8th-order filter is expressed as a single high-order denominator polynomial:

$$A(z) = 1 + a_1 z^{-1} + a_2 z^{-2} + \ldots + a_8 z^{-8}$$

The roots of the polynomial (the poles d_k) are extraordinarily sensitive to microscopic floating-point rounding errors in the coefficients a_k. In fact, a tiny 0.0001% quantization error in coefficient a_5 can cause poles near the unit circle to migrate OUTSIDE the unit circle ($|d_k| > 1.0$), transforming a stable filter into an explosive, oscillating amplifier that blows up to NaN!

To prevent numerical explosion, **Second-Order Sections (SOS)** factorizes the 8th-order rational transfer function into a cascade of four independent 2nd-order biquadratic filter blocks:

Cascade Decomposition into Second-Order Sections (SOS / Biquads)

$$H(z) = \prod_{i=1}^{L} H_i(z) = \prod_{i=1}^{4} \frac{b_{0i} + b_{1i} z^{-1} + b_{2i} z^{-2}}{1 + a_{1i} z^{-1} + a_{2i} z^{-2}}$$

Each 2nd-order biquad handles only a single conjugate pair of poles and zeros. Because the roots are isolated to 2nd-degree polynomials, root sensitivity is minimized, guaranteeing rock-solid stability even under 64-bit IEEE floating-point arithmetic.

5.4 The 50 Hz IIR Notch Filter: Surgical Unit-Circle Zeros

To eliminate 50 Hz AC mains hum, Raima designed an accompanying IIR notch filter using `signal.iirnotch(50.0, 30.0, fs=self.fs)`.

The transfer function of the digital notch filter is given by:

Biquad Transfer Function of the 50 Hz IIR Notch Filter

$$H_{notch}(z) = \frac{1 - 2\cos(\Omega_0) z^{-1} + z^{-2}}{1 - 2 r \cos(\Omega_0) z^{-1} + r^2 z^{-2}}$$

Where $\Omega_0 = 2\pi$ (50.0 / 1000.0) = 0.1π rad/sample, and the pole radius r is derived from the Quality Factor (Q = 30.0):

Pole Radius Calculation for Razor-Thin Notch Bandwidth

$$BW = \frac{f_0}{Q} = \frac{50.0}{30.0} \approx 1.667\,\text{Hz} \implies r \approx 1 - \pi \left(\frac{BW}{f_s}\right) = 1 - \pi \left(\frac{1.667}{1000}\right) \approx 0.9947$$

Because the zeros lie exactly on the unit circle ($|z_{zero}| = 1.0$), the frequency response magnitude drops to **absolute zero (−∞ dB) at exactly 50.0 Hz**. And because the poles lie at r = 0.9947 directly behind the zeros, the gain snaps back to unity (0 dB) at 48 Hz and 52 Hz!

5.5 Python Implementation Walkthrough & State Management

Let us examine the exact production Python code written by Raima Ghosh inside `emg_methods.py`:

emg_methods.py: Raima's Complete IIR Streaming Implementation

```
# 1. IIR (Raima) - Filter Initialization in __init__
high = min(450.0, 0.45 * self.fs)
self.iir_sos = signal.butter(4, (20.0, high), btype='bandpass', fs=self.fs, output='sos')
self.notch_b, self.notch_a = signal.iirnotch(50.0, 30.0, fs=self.fs)
self.zi_bp = None
self.zi_notch = np.zeros(max(len(self.notch_a), len(self.notch_b)) - 1)

# Streaming Process Method:
def process(self, raw_uv):
    x = np.asarray(raw_uv, dtype=float)
    if self.method == 'IIR':
        if self.zi_bp is None:
            # Cold-start transient mitigation: initialize delay registers to steady-state
            self.zi_bp = signal.sosfilt_zi(self.iir_sos) * x[0]
        # Step 1: Filter raw microvolts through 4-stage SOS bandpass
        y, self.zi_bp = signal.sosfilt(self.iir_sos, x, zi=self.zi_bp)
        # Step 2: Cascade filtered output through 50 Hz IIR notch filter
        y, self.zi_notch = signal.lfilter(self.notch_b, self.notch_a, y, zi=self.zi_notch)
    self.history.extend(y)
    return y
```

Step-by-Step Code Autopsy:

- `signal.butter(4, (20.0, high), btype='bandpass', fs=self.fs, output='sos')`: Generates the 4th-order (8-pole) bandpass SOS matrix. The matrix has dimensions (4, 6), representing four biquad sections with 6 coefficients each (b_0, b_1, b_2, 1, a_1, a_2).
- `signal.sosfilt_zi(self.iir_sos) * x[0]`: Solves the steady-state initial condition vectors. If you start a filter with zeros in its delay registers while the input voltage is at +500 µV, the filter interprets this as a violent 500-volt step function, creating a ringing transient spike that lasts for 100 ms! Multiplying the unit step response by the very first sample x[0] pre-charges the internal registers, ensuring immediate, smooth streaming from sample zero.
- `signal.sosfilt(..., zi=self.zi_bp)`: Executes causal second-order section filtering on the 20-sample chunk, returning both the filtered output y and the updated state vector z_{i_bp}. This state vector is preserved across iterations.
- `signal.lfilter(self.notch_b, self.notch_a, y, zi=self.zi_notch)`: Direct form II transposed filtering of the notch stage, maintaining notch delay memory in `zi_notch`.

5.6 Latency and Group Delay Analysis

The only minor engineering compromise of the Butterworth IIR filter is its **non-linear phase response**. The derivative of phase with respect to frequency is the **Group Delay**:

Definition of Filter Group Delay (Envelope Delay)

$$\tau_g(\omega) = -\frac{d\theta(\omega)}{d\omega}$$

In Raima's Butterworth bandpass filter, the group delay peaks near the 20 Hz lower cutoff frequency (≈ 25 ms delay) and tapers to approximately 5 to 8 ms in the dominant 100 Hz muscle region.

Cascaded with the 50 Hz notch filter, the aggregate group delay is approximately **35 to 45 milliseconds**. In an interactive VR weightlifting game, human conscious visual-motor response latency is on the order of 150 to 200 ms. A filter latency of 40 ms is virtually imperceptible to the athlete, delivering an instantaneous, natural feeling of muscle-activated virtual lifting!

PART III: THE FOUR FILTER SUPERHEROES — DEEP TECHNICAL ANALYSIS • CHAPTER 6

Finite Impulse Response (FIR) Filter — Kunal's Method

Exploring non-recursive feedforward architectures, strict linear phase preservation, window-based filter design via the Hamming window, and unconditional numerical stability.

6.1 Non-Recursive Architecture and Finite Impulse Response (FIR)

The second filtering methodology in our comparative project was architected and implemented by student researcher **Kunal Kashyap**. Kunal took a completely different philosophical approach to linear systems design, championing the **Finite Impulse Response (FIR) Digital Filter**.

In stark contrast to Raima's recursive IIR filter, an FIR filter contains **zero feedback loops**. The current output sample y[n] is computed exclusively as a weighted linear combination of the current input sample and past input samples:

The General Difference Equation for an M-th Order FIR Filter

$$y[n] = \sum_{k=0}^{M} b_k \cdot x[n-k] = b_0\, x[n] + b_1\, x[n-1] + b_2\, x[n-2] + \dots + b_M\, x[n-M]$$

Where b_k are the *filter coefficients (filter taps)*, and M is the filter order (the number of taps is $N_{taps} = M + 1$). In feedback notation, all denominator coefficients a_k (for $k \geq 1$) are identically zero.

> 💡 **Intuitive Analogy: The Weighted Conveyor Belt**
>
> Imagine an industrial conveyor belt carrying past numbers on cards. The conveyor belt has 101 positions marked out. Every millisecond, the conveyor belt shifts forward by one position: a brand new number x[n] lands on position 0, while the oldest number at position 100 falls off the end into the recycling bin.
>
> Above each position sits a stationary magnifying lens that multiplies the number on that card by a fixed weight b_k. A grand digital summing scale adds up all 101 multiplied numbers at once to produce the output y[n].
>
> If you feed a single 1 on card 0 and zeros everywhere else (a unit impulse δ[n]), the 1 travels down the 101 positions of the belt. For 101 milliseconds, you read out the weights b_0, b_1, …, b_{100}. But on millisecond 102, the card drops off the belt, and the output becomes **EXACTLY ZERO** forever! That is why this filter is called a **Finite Impulse Response**!

Unconditional Numerical Stability: The Magic of All-Zero Poles

The greatest theoretical advantage of Kunal's FIR architecture is that it is **unconditionally stable under all operating conditions**.

In z-transform analysis, the transfer function of an FIR filter is:

FIR Transfer Function Form: All M Poles Reside at the Origin

$$H(z) = \sum_{k=0}^{M} b_k\, z^{-k} = \frac{b_0\, z^M + b_1\, z^{M-1} + \dots + b_M}{z^M}$$

Notice the denominator polynomial: it is simply z^M. This means that **all M poles of an FIR filter reside exclusively at z = 0 (the origin of the complex z-plane)**!

Because all poles sit safely at the origin—infinitely far away from the boundary of the unit circle (|z| = 1.0)—an FIR filter can NEVER become unstable, can never oscillate out of control, and can never blow up to NaN, regardless of numerical truncation, coefficient quantization, or extreme signal surges!

6.2 Linear Phase Property and Morphological Waveform Fidelity

Why would Kunal accept the higher computational cost of 101 taps instead of Raima's 4th-order IIR? The answer lies in one supreme mathematical property: **Strict Linear Phase**.

If the coefficients of an FIR filter are designed with even symmetry about their midpoint:

Symmetric Coefficient Condition for Type I Linear-Phase FIR Filters

$$b_k = b_{M-k} \quad \text{for } k = 0, 1, \dots, M$$

Then the frequency response of the filter possesses a phase angle that is a perfectly straight, linear function of digital frequency:

Strict Linear Phase Response as a Function of Frequency

$$\angle H(e^{j\Omega}) = -\frac{M}{2} \cdot \Omega$$

Constant Group Delay Across All Frequencies

Taking the negative derivative of this linear phase function yields a constant:

Constant Group Delay of Kunal's 101-Tap FIR Filter

$$\tau_g = -\frac{d\angle H(e^{j\Omega})}{d\Omega} = \frac{M}{2}\,\text{samples} = \frac{101 - 1}{2} = 50\,\text{samples}$$

At a sampling frequency of f_s = 1000 Hz (where each sample is exactly 1.0 ms), a group delay of 50 samples means that **EVERY SINGLE FREQUENCY COMPONENT is delayed by exactly 50.0 milliseconds**!

🔬 **Technical Fact & Definition: Why Linear Phase Preserves MUAP Waveforms**

In non-linear phase filters (like Butterworth IIR), a 40 Hz wave component might be delayed by 25 ms while a 150 Hz component is delayed by only 6 ms. This unequal delay causes **Phase Distortion**: the individual frequency components drift apart in time, dispersing and smearing the sharp spike of a Motor Unit Action Potential.

In Kunal's FIR filter, because every frequency is delayed by exactly 50.0 ms, the biological waveform is shifted in time as a rigid, cohesive block without any dispersion! The exact peaks, valleys, and morphological features of the biceps action potential are preserved with 100% fidelity.

6.3 FIR Filter Design via Windowing (Lathi Section 7.8)

How did Kunal calculate the 101 numerical values for coefficients b_0, b_1, ..., b_{100}? He used the classical **Window Design Method**.

📖 **B.P. Lathi Textbook Reference (Section 7.8): The Window Method for FIR Filter Design**

In Section 7.8 of *Linear Systems and Signals*, B.P. Lathi explains the fundamental Fourier duality governing FIR filter design:

An ideal bandpass filter has a rectangular frequency response with sharp vertical drop-offs. Taking the Inverse Discrete-Time Fourier Transform (IDTFT) of this ideal rectangular frequency response yields an impulse response that is a two-sided, infinite **sinc function**:

$$h_{ideal}[n] = [\sin(\omega_2\, n) - \sin(\omega_1\, n)] \,/\, (\pi\, n) \qquad (-\infty < n < \infty)$$

Because an infinite sinc function cannot be computed in finite time, we must truncate it. But if you truncate it abruptly with a sharp rectangular window, Fourier analysis dictates that the abrupt discontinuity creates violent **Gibbs Phenomenon ringing** with stopband sidelobes of only −13 dB!

To suppress Gibbs oscillations, Lathi shows that we must multiply the ideal sinc response by a smooth, tapered **Window Function w[n]** that gently eases to zero at both edges.

The Hamming Window Formulation

Kunal chose the renowned **Hamming Window**, defined mathematically as:

Mathematical Formulation of the Hamming Window (M = 100)

$$w[n] = 0.54 - 0.46 \cdot \cos\left(\frac{2\pi n}{M} \right) \quad \text{for } 0 \le n \le M$$

The coefficients of Kunal's FIR filter are the direct point-by-point product of the ideal continuous sinc function and the Hamming window:

Truncation and Windowing Equation for Causal FIR Coefficients

$$b_k = h_{ideal}[k - M/2] \cdot w[k] \quad \text{for } k = 0, 1, \dots, M$$

The Hamming window suppresses stopband sidelobes down to **−53 dB** relative to the passband peak! This guarantees that out-of-band motion artifacts below 20 Hz and high-frequency thermal noise above 450 Hz are attenuated by a factor exceeding **400 to 1**.

Why Exactly 101 Taps? The Transition Bandwidth Trade-off

For a Hamming-windowed FIR filter, the transition bandwidth Δf between passband and stopband is governed by the number of taps:

Transition Bandwidth Resolution of Kunal's 101-Tap Filter

$$\Delta f \approx 3.3 \cdot \frac{f_s}{N_{taps}} = 3.3 \cdot \frac{1000\,\text{Hz}}{101} \approx 32.7\,\text{Hz}$$

With N_{taps} = 101, the filter transitions from passband to full stopband attenuation in approximately 33 Hz. Increasing to 301 taps would make the transition sharper (11 Hz), but it would triple the group delay to 150 ms, introducing unacceptable visual lag into the VR weightlifting game! A tap count of 101 provides the perfect sweet spot between frequency selectivity and real-time responsiveness.

6.4 Python Implementation Walkthrough

Let us examine the exact production Python code written by Kunal Kashyap inside `emg_methods.py`:

emg_methods.py: Kunal's Complete FIR Streaming Implementation

```
 # 2. FIR (Kunal) - Filter Initialization in __init__
numtaps = 101
high = min(450.0, 0.45 * self.fs)
# Design 101-tap linear-phase bandpass filter using Hamming window
self.fir_b = signal.firwin(numtaps, [20.0, high], pass_zero=False, fs=self.fs)
# State vector: internal delay line for FIR convolution (length = numtaps - 1)
self.fir_zi = np.zeros(len(self.fir_b) - 1)
# State vector for cascaded 50 Hz IIR notch filter
self.zi_fir_notch = np.zeros(max(len(self.notch_a), len(self.notch_b)) - 1)

# Streaming Process Method:
def process(self, raw_uv):
    x = np.asarray(raw_uv, dtype=float)
    if self.method == 'FIR':
        # Step 1: Convolve incoming chunk with 101-tap FIR weights, preserving fir_zi
        y, self.fir_zi = signal.lfilter(self.fir_b, 1.0, x, zi=self.fir_zi)
        # Step 2: Cascade through 50 Hz notch filter to eradicate power-line hum
        y, self.zi_fir_notch = signal.lfilter(self.notch_b, self.notch_a, y, zi=self.zi_fir_notch)
    self.history.extend(y)
    return y
```

Step-by-Step Code Autopsy:

- `pass_zero=False`: This parameter is critical. If `pass_zero=True`, SciPy would create a band-stop or low-pass filter where 0 Hz (DC) is passed! Setting `pass_zero=False` forces 0 Hz into the stopband, ensuring the filter functions as a true bandpass that destroys DC baseline wander.
- `self.fir_zi = np.zeros(len(self.fir_b) - 1)`: An array of 100 zeros that stores the most recent 100 samples from the previous 20 ms chunk. This ensures seamless mathematical continuity across packet boundaries.
- `signal.lfilter(self.fir_b, 1.0, x, zi=self.fir_zi)`: Because the denominator is simply `1.0`, `lfilter` executes pure feedforward FIR convolution.

6.5 Comprehensive IIR vs FIR Comparison Matrix

To synthesize the engineering trade-offs between Raima's IIR and Kunal's FIR methods, we present an exhaustive comparison across all technical dimensions:

Engineering Dimension	Raima's Method (Butterworth IIR)	Kunal's Method (Hamming FIR)
Mathematical Structure	Recursive feedback: $y[n] = \Sigma\, b_k\, x[n-k] - \Sigma\, a_k\, y[n-k]$	Pure feedforward: $y[n] = \Sigma\, b_k\, x[n-k]$
Numerical Stability	Conditionally stable; poles must lie strictly inside $\|z\| < 1$ (stabilized by SOS)	**Unconditionally stable**; all 100 poles reside at $z = 0$
Phase Linearity	Non-linear phase; higher frequencies experience different delays	**Strict linear phase**; constant group delay across all frequencies
Waveform Distortion	Slight phase dispersion of sharp biological MUAP spikes	**Zero dispersion**; perfect morphological shape preservation
Group Delay (Latency)	Dynamic: 5–25 ms in passband (≈ 35–45 ms total)	Constant: exactly 50 samples = **50.0 ms**
Computational Cost	Ultra-low: 4 biquads = 16 multiplies + 16 adds per sample	Moderate: 101 multiplies + 100 adds per sample (approx 6× higher)
Memory Footprint	8 floating-point state variables	100 floating-point delay registers
Best Application in VR	Fast, snappy responsiveness where lowest latency is paramount	High-precision biomechanical research where signal fidelity is king

PART III: THE FOUR FILTER SUPERHEROES — DEEP TECHNICAL ANALYSIS • CHAPTER 7

Fast Fourier Transform (FFT) Filter — Chandan's Method

Deconstructing discrete-time Fourier analysis, the Cooley-Tukey FFT algorithm, block spectral masking, time-frequency duality, and circular convolution dynamics.

7.1 The Fourier Transform and Time-Frequency Duality

The third filtering methodology in our comparative study was architected and implemented by student researcher **Chandan Kumar**. Chandan took an audacious, frequency-first approach: instead of applying difference equations in the time domain (like Raima and Kunal), he transformed the electromyographic signal directly into the **frequency domain** using the **Fast Fourier Transform (FFT)**, applied a surgical binary mask, and transformed back into the time domain!

> 📖 **B.P. Lathi Textbook Reference (Chapter 7): The Fourier Transform & Time-Frequency Duality**
> In Chapter 7 of *Linear Systems and Signals*, B.P. Lathi presents one of the most profound mathematical discoveries in human history, first formulated by Jean-Baptiste Joseph Fourier in 1822:
>
> *Any physical signal, no matter how irregular or chaotic it appears in the time domain, can be uniquely decomposed into an infinite sum of pure sinusoidal waves, each possessing a distinct frequency, amplitude, and phase angle.*
>
> Time and frequency are dual representations of the exact same physical reality. A sharp transient spike in the time domain spreads out across all frequencies; conversely, an isolated single-frequency tone in the frequency domain extends across all time.

> 💡 **Intuitive Analogy: Unbaking a Cake into Raw Ingredients**
> Imagine someone hands you a finished chocolate birthday cake. In the **Time Domain**, all you see is a dark, delicious, solid cake.
>
> Now imagine you have a magical laboratory machine: you insert a slice of cake, and the machine instantly separates it into neat little glass jars sitting on a table: 200 grams of flour, 100 grams of sugar, 50 grams of cocoa powder, and 1 teaspoon of bitter spoiled salt (50 Hz noise)!
>
> That magical separation machine is the **Fourier Transform**! Now you can simply pick up the jar of spoiled salt, throw it in the trash (spectral masking), and run the machine in reverse (Inverse Fourier Transform) to reconstitute a pure, perfect chocolate cake! That is exactly what Chandan's filter does to the biceps signal!

7.2 The Discrete Fourier Transform (DFT)

To perform Fourier analysis on a digital computer, we must sample both the time domain and the frequency domain. This discrete formulation is the **Discrete Fourier Transform (DFT)**.

For a finite block of N discrete time-domain samples x[0], x[1], ..., x[N-1], the DFT computes a sequence of N complex frequency coefficients X[k]:

The Discrete Fourier Transform (DFT) Analysis Equation

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j \frac{2\pi}{N} k n} = \sum_{n=0}^{N-1} x[n] \left[\cos\left(\frac{2\pi k n}{N}\right) - j \sin\left(\frac{2\pi k n}{N}\right) \right]$$

Where $k \in \{0, 1, ..., N-1\}$ is the discrete frequency bin index. Each complex number X[k] contains:

- **Magnitude $|X[k]| = \sqrt{(Re(X[k])^2 + Im(X[k])^2)}$:** The amplitude of the sinusoid oscillating at frequency f_k.
- **Phase $\angle X[k] = \arctan2(Im(X[k]), Re(X[k]))$:** The starting time offset of that sinusoid.

Frequency Bin Resolution for Chandan's Buffer (N = 256)

The physical frequency corresponding to bin index k is given by:

Frequency Bin Spacing of Chandan's 256-Point FFT

$$f_k = k \cdot \frac{f_s}{N} = k \cdot \frac{1000\,\text{Hz}}{256} = k \cdot 3.90625\,\text{Hz}$$

Each discrete bin represents a frequency slice of approximately **3.91 Hz**. For example:

- Bin k = 0 → 0.0 Hz (Pure DC baseline offset).
- Bin k = 5 → 5 × 3.90625 = 19.53 Hz (Boundary of motion artifacts).
- Bin k = 13 → 13 × 3.90625 = 50.78 Hz (The 50 Hz power-line interference bin!).
- Bin k = 115 → 115 × 3.90625 = 449.22 Hz (Upper boundary of muscle activity).
- Bin k = 128 → 128 × 3.90625 = 500.0 Hz (The Nyquist folding limit, $f_s/2$).

7.3 The Fast Fourier Transform (FFT) Algorithm

If you evaluate the DFT equation directly using brute-force matrix multiplication, computing N complex frequency coefficients requires $N \times N = N^2$ complex multiplications. For large data blocks, this quadratic scaling rapidly overwhelms the CPU.

In 1965, J.W. Cooley and John Tukey published the **Fast Fourier Transform (FFT)** algorithm. By recognizing that the complex twiddle factors $W_N^{kn} = e^{-j\ 2\pi\ kn\ /\ N}$ possess periodic and symmetric symmetries ($W_N^{k\ +\ N/2} = -W_N^{k}$), Cooley and Tukey recursively split an N-point DFT into two N/2-point DFTs (one containing the even-indexed samples, the other containing the odd-indexed samples).

Computational Complexity Reduction: 32× Acceleration via Radix-2 FFT

$$\text{Brute Force DFT: } \mathcal{O}(N^2) = 256^2 = 65{,}536\,\text{operations} \quad\longrightarrow\quad \text{Cooley-Tukey FFT: } \mathcal{O}(N \log_2 N) = 256 \times 8 = 2{,}048\,\text{operations}$$

The Real FFT Optimization (`np.fft.rfft`)

Because the raw electromyographic signal from the BioRadio is composed purely of real numbers (zero imaginary component), the resulting Fourier spectrum exhibits **Hermitian Conjugate Symmetry**:

Hermitian Symmetry of Real-Valued Discrete Signals

$$X[N - k] = X^{*}[k]$$

The upper half of the frequency spectrum (from 500 Hz to 1000 Hz) is a redundant, mirror-image conjugate of the lower half (from 0 to 500 Hz). In NumPy, Chandan leveraged `np.fft.rfft`, which computes only the non-redundant frequencies from 0 up to the Nyquist limit (N/2 + 1 = 129 complex bins). This halves both memory allocation and execution time!

7.4 Block Spectral Masking Implementation Walkthrough

Let us examine the exact production Python code written by Chandan Kumar inside `emg_methods.py`:

emg_methods.py: Chandan's Complete FFT Streaming Implementation

```
# 3. FFT (Chandan) - Initialization in __init__
self.fft_buf = deque(maxlen=256)

# Streaming Process Method:
def process(self, raw_uv):
    x = np.asarray(raw_uv, dtype=float)
    if self.method == 'FFT':
        # Step 1: Append incoming 20-sample chunk into sliding 256-sample ring buffer
        self.fft_buf.extend(x)
        buf = np.array(self.fft_buf)
        N = len(buf)
        if N >= 32:
            # Step 2: Compute real frequency axis and complex Fourier spectrum
            freqs = np.fft.rfftfreq(N, d=1.0 / self.fs)
            X = np.fft.rfft(buf)
            # Step 3: Create binary Boolean mask (Bandpass 20-450 Hz + Notch 50 Hz +/- 2 Hz)
            mask = (freqs >= 20.0) & (freqs <= 450.0) & (np.abs(freqs - 50.0) > 2.0)
            # Step 4: Multiply spectrum by mask (zero out forbidden frequencies)
            X_filt = X * mask
            # Step 5: Synthesize clean time-domain signal via Inverse Real FFT
            y_full = np.fft.irfft(X_filt, n=N)
            # Step 6: Overlap extraction - slice out the most recent chunk
            y = y_full[-len(x):]
        else:
            y = x
    self.history.extend(y)
    return y
```

Step-by-Step Code Autopsy:

- `deque(maxlen=256)` : A circular double-ended queue that maintains a rolling history of the most recent 256 samples (256 milliseconds at 1000 Hz).
- `freqs = np.fft.rfftfreq(N, d=1.0/self.fs)` : Generates the exact 129 frequency values corresponding to each bin (0.0 Hz, 3.91 Hz, 7.81 Hz, ..., 500.0 Hz).
- `mask = (freqs >= 20.0) & (freqs <= 450.0) & (np.abs(freqs - 50.0) > 2.0)` : A boolean array where bins are `True` (1) inside the valid band and `False` (0) below 20 Hz, above 450 Hz, or within 2 Hz of 50 Hz.
- `X_filt = X * mask` : Zeroes out forbidden frequency bins. Frequencies in bins 0 through 4 (DC and motion artifacts), bin 13 (50 Hz hum), and bins 116 through 128 (high-frequency noise) are forced to 0.0 + 0.0j.
- `y_full = np.fft.irfft(X_filt, n=N)` : Reconstructs the 256-sample time-domain block via the Inverse Real FFT.
- `y = y_full[-len(x):]` : Because only 20 new samples arrived in this update cycle, Chandan extracts the most recent 20 samples from the reconstructed block, matching the streaming interface!

7.5 Trade-offs, Circular Convolution, and Spectral Leakage

Chandan's FFT filter offers surgical precision, but it carries profound theoretical consequences that must be understood:

> 📖 **B.P. Lathi Textbook Reference (Section 7.3 & 9.4): Multiplication in Frequency = Circular Convolution in Time**
> In Section 7.3 and 9.4 of *Linear Systems and Signals*, B.P. Lathi establishes the **Convolution-Multiplication Duality**:
>
> Multiplying two discrete spectra X[k] · M[k] in the discrete frequency domain does NOT correspond to ordinary linear convolution in time; it corresponds to **Circular (Cyclic) Convolution**!
>
> In circular convolution, samples that trail off the right edge of the block wrap around and corrupt the left edge of the block (time-domain aliasing). Furthermore, multiplying by a sharp rectangular mask in frequency creates an effective sinc response in time, which can introduce subtle pre-echo ringing artifacts before explosive muscle bursts.

Feature / Metric	Chandan's FFT Filter	Raima's IIR Filter	Kunal's FIR Filter
Processing Domain	Discrete Frequency Domain	Discrete Time Domain	Discrete Time Domain
50 Hz Rejection Precision	Surgical bin zeroing (±2 Hz)	IIR Notch biquad (BW ≈ 1.67 Hz)	Cascaded IIR Notch biquad
Internal Buffer Size	256 samples (256 ms window)	4 state variables	100 delay registers
Effective Latency	Moderate to High (~128 ms centroid)	Low (≈ 35–45 ms)	Fixed: exactly 50.0 ms
Spectral Leakage Sensitivity	High (rectangular window leakage)	None (continuous streaming)	Suppressed by Hamming window
Key Pedagogical Strength	Direct visualization of frequency bins	Extreme computational efficiency	Guaranteed stability & linear phase

PART III: THE FOUR FILTER SUPERHEROES — DEEP TECHNICAL ANALYSIS • CHAPTER 8

First-Order Difference (DIFF) Filter — Krishna's Method

Analyzing discrete-time numerical differentiation as an ultra-fast high-pass operator, its frequency response magnitude, noise amplification trade-offs, and streaming state bridging.

8.1 Differentiation as a First-Principles High-Pass Filter

The fourth and final filtering methodology in our comparative project was architected and implemented by student researcher **Krishna Patel**. Krishna embraced a minimalist, first-principles philosophy: what is the simplest, fastest, and lowest-latency mathematical operation that can remove DC baseline wander and detect muscle contraction onsets?

The answer is **Numerical Differentiation**. In the continuous-time domain, the derivative of a signal x(t) with respect to time represents its instantaneous rate of change:

Continuous-Time Differentiation in Time and Laplace Domains

$$y(t) = \frac{dx(t)}{dt} \quad \Longleftrightarrow \quad Y(s) = s \cdot X(s)$$

In the discrete-time domain, the continuous derivative is approximated by the **First-Order Backward Difference Operator**:

The First-Order Discrete Difference Equation

$$y[n] = x[n] - x[n-1]$$

Notice the astonishing simplicity of this formula: computing y[n] requires **exactly one subtraction operation** per sample! There are no floating-point multiplications, no trigonometric sine or cosine lookups, no polynomial divisions, and no matrix transforms.

> 💡 **Intuitive Analogy: The Sports Car Speedometer**
>
> Imagine you are riding in a car whose altitude is constantly changing as it climbs up and down gentle rolling hills (DC baseline drift and slow motion artifacts < 20 Hz).
>
> If you look out the window at the elevation markers, your absolute altitude is swinging by hundreds of meters.
>
> Now look down at the **Speedometer**! The speedometer does not care what altitude you are at; it only measures how fast your position is changing right now (the derivative)! If the car is driving smoothly at constant elevation, the derivative is zero. But the second you step on the gas pedal and accelerate violently (a sudden muscle contraction burst), the speedometer shoots upward instantly! Krishna's difference filter acts as an electrophysiological speedometer for your arm!

8.2 Frequency Response of the Difference Operator

To understand how a simple subtraction functions as a digital filter, we analyze its z-transform and frequency response.

Applying the time-shifting property of the z-transform to the difference equation y[n] = x[n] − x[n-1]:

Transfer Function of the First-Order Difference Filter

$$Y(z) = X(z) - z^{-1} X(z) = (1 - z^{-1}) X(z) \implies H(z) = 1 - z^{-1} = \frac{z - 1}{z}$$

Examining the pole-zero constellation of H(z):

- **One Zero at z = 1.0:** Because z = 1.0 = $e^{j\,0}$ corresponds to Ω = 0 rad/sample (0.0 Hz / Pure DC), **the filter has a transmission zero placed dead center at DC**! The gain at 0 Hz is identically zero, completely annihilating static electrode half-cell voltages and slow baseline drift.
- **One Pole at z = 0.0:** The single pole sits harmlessly at the origin, ensuring unconditional FIR stability.

Derivation of Magnitude and Phase Response

Evaluating H(z) on the unit circle where z = $e^{j\Omega}$:

Analytical Frequency Response Derivation on the Unit Circle

$$H(e^{j\Omega}) = 1 - e^{-j\Omega} = e^{-j\Omega/2} \left(e^{j\Omega/2} - e^{-j\Omega/2} \right) = 2j \cdot \sin\left(\frac{\Omega}{2}\right) \cdot e^{-j\Omega/2}$$

Recalling that $j = e^{j\pi/2}$, we separate the frequency response into its magnitude and phase components:

Magnitude Response and Linear Phase of Krishna's Differentiator

$$|H(e^{j\Omega})| = 2 \left| \sin\left(\frac{\Omega}{2}\right) \right| \quad \text{and} \quad \angle H(e^{j\Omega}) = \frac{\pi}{2} - \frac{\Omega}{2}$$

🔬 **Technical Fact & Definition: Linear High-Pass Gain Characteristics**

For small digital frequencies ($\Omega \ll 1$), the Taylor expansion yields $\sin(\Omega/2) \approx \Omega/2$. Therefore:

$$|H(e^{j\Omega})| \approx 2 \cdot (\Omega / 2) = \Omega = 2\pi f / f_s$$

The magnitude response is directly proportional to frequency! On a logarithmic Bode plot, the gain rises at a steady **+20 dB per decade (+6 dB per octave)**.

This confirms that the difference operator is inherently a **High-Pass Filter**: low frequencies are squashed, while high frequencies pass through with increasing amplification.

8.3 Strengths and Hazards: Muscle Onset Detection vs Noise Explosion

Krishna's difference filter exhibits dramatic strengths and severe hazards that make it a fascinating study in engineering trade-offs:

Microscopic Latency (0.5 ms) — Supreme Virtue

The group delay is $\tau_g = -d\theta/d\Omega$ = **0.5 samples = 0.5 milliseconds**! This is the fastest possible response in discrete-time math. Explosive bicep contractions are detected with zero perceptible delay.

Zero Computational Overhead — Efficiency

Requires only a single subtraction per sample. It can execute on a microscopic 8-bit microcontroller with zero floating-point hardware.

High-Frequency Noise Amplification — Deadly Hazard

Because the gain rises with frequency, high-frequency noise is amplified! At 400 Hz ($\Omega = 0.8\pi$), the gain is $2 \cdot \sin(0.4\pi)$ = **1.902 (+5.6 dB)**, doubling high-frequency noise!

Sensitivity to 50 Hz Mains Hum — Catastrophic Flaw

At 50 Hz ($\Omega = 0.1\pi$), the gain is $2 \cdot \sin(0.05\pi) = 0.3128$. While attenuated relative to 400 Hz, power-line hum still leaks through with significant energy if not notched.

⚠️ **Critical Pitfall & Warning: Why the 50 Hz Notch Filter is Mandatory for DIFF**

Because the differentiator amplifies higher frequencies, **any ambient 50 Hz power-line interference or 100/150 Hz harmonics will be dramatically boosted relative to baseline**!

To prevent Krishna's method from turning into an uncontrollable 50 Hz buzzing generator, Krishna cascaded Raima's razor-sharp 50 Hz IIR notch filter immediately following the difference step: `y, self.zi_notch = signal.lfilter(self.notch_b, self.notch_a, y, zi=self.zi_notch)`. This marriage of a 1st-order differentiator with an IIR notch produces an effective, low-latency onset detector.

8.4 Streaming Implementation & Chunk Boundary Continuity

Let us examine the exact production Python code written by Krishna Patel inside `emg_methods.py`:

emg_methods.py: Krishna's Complete DIFF Streaming Implementation

```
# 4. DIFF (Krishna) - Initialization in __init__
self.diff_prev = None
# Shares Raima's 50 Hz notch filter state
self.zi_notch = np.zeros(max(len(self.notch_a), len(self.notch_b)) - 1)

# Streaming Process Method:
def process(self, raw_uv):
    x = np.asarray(raw_uv, dtype=float)
    if self.method == 'DIFF':
        # Cold start handling: if very first packet, initialize previous sample
        if self.diff_prev is None:
            self.diff_prev = x[0]
        # Step 1: Prepend the last sample of previous chunk to avoid losing 1st diff
        extended = np.insert(x, 0, self.diff_prev)
        # Step 2: Compute first-order backward differences: y[n] = extended[n] - extended[n-1]
        y = np.diff(extended)
        # Step 3: Cache the final sample of current chunk for next packet boundary
        self.diff_prev = x[-1]
        # Step 4: Pass differentiated signal through 50 Hz notch filter
        y, self.zi_notch = signal.lfilter(self.notch_b, self.notch_a, y, zi=self.zi_notch)
    self.history.extend(y)
    return y
```

The Chunk Boundary Problem and the `diff_prev` Solution

When computing discrete differences on an array of length L, the standard `np.diff(x)` function produces an array of length L − 1:

The Missing Sample Dilemma of Naive Differentiation

$$\text{len}(x) = 20 \implies \text{len}(\text{np.diff}(x)) = 19$$

If you drop one sample every 20 ms update cycle, your system loses 50 samples every second! Furthermore, the difference between the first sample of chunk k and the last sample of chunk k−1 would never be computed, creating a discontinuity.

Krishna solved this elegantly:

1. In `self.diff_prev`, he caches the very last sample of the preceding chunk ($x_{k-1}[19]$).
2. Using `np.insert(x, 0, self.diff_prev)`, he prepends this cached sample to the front of the incoming 20-sample chunk, creating an extended array of length 21.
3. Calling `np.diff(extended)` computes all 20 differences: $\Delta x[0] = x_k[0] - x_{k-1}[19]$, $\Delta x[1] = x_k[1] - x_k[0]$, ..., $\Delta x[19] = x_k[19] - x_k[18]$.
4. The output array y has length exactly 20, preserving seamless continuity across chunk boundaries without a single missing sample!

PART IV: FROM FILTERED SIGNAL TO MUSCLE POWER NUMBER • CHAPTER 9

Rectification, Root Mean Square (RMS), and the Linear Envelope

Translating filtered alternating microvolt fluctuations into a smooth, unipolar physical power metric that faithfully tracks continuous muscle force.

9.1 The Rectification Imperative: Escaping the Zero-Sum Trap

After passing raw biceps voltages through Raima's IIR, Kunal's FIR, Chandan's FFT, or Krishna's DIFF filter, we possess a clean, purified signal. Motion artifacts below 20 Hz have been eradicated, 50 Hz power-line interference has been destroyed, and high-frequency thermal hiss above 450 Hz has been suppressed.

Yet, if we take this beautifully filtered signal and plot its mathematical average, we confront an unavoidable biological reality:

The Persistent Zero-Mean Property of Purified Electromyographic Signals

$$\text{Mean}(y_{filtered}[n]) = \frac{1}{N} \sum_{n=0}^{N-1} y_{filtered}[n] \approx 0.00\,\mu\text{V}$$

Because Motor Unit Action Potentials are fundamentally biphasic electrical dipoles that oscillate symmetrically above and below zero volts, **filtering the noise does not change the fact that positive swings cancel negative swings**.

If we want a number that can command a virtual reality character to curl a dumbbell, we must convert bidirectional electrical oscillations into a strictly positive, unipolar signal representing **instantaneous biological effort**. This mathematical conversion begins with **Rectification**:

Full-Wave Rectification (Selected) `|y[n]|`

Inverts every negative voltage swing into an identical positive voltage swing: $y_{rect}[n] = |y[n]|$. Preserves 100% of the muscle's electrical energy and doubles the effective pulse frequency.

Half-Wave Rectification (Rejected) `max(0, y[n])`

Sets all negative voltage swings to zero: $y_{rect}[n] = \max(0, y[n])$. Discards 50% of the biological data, severely degrading Signal-to-Noise Ratio.

💡 **Intuitive Analogy: The Windmill Analogy**

Imagine an oscillating storm wind that blows violently to the North for 1 second, then reverses and blows violently to the South for 1 second, alternating back and forth 100 times every second.

If you measure the *net movement of air molecules* over an hour, the average wind displacement is ZERO. But if you place a windmill in that wind, the windmill's blades spin furiously! The mechanical gears of the windmill don't care which direction the wind blows; they rectify the motion and extract rotational kinetic power.

Rectification and RMS are the digital windmill of our VR game: they extract pure muscle power regardless of whether the electrical wave swung positive or negative!

9.2 The Mathematical Anatomy of Root Mean Square (RMS)

While simple full-wave rectification (|y[n]|) solves the polarity problem, the rectified signal remains an intensely jagged, rapidly spiking waveform that jumps between 0 µV and 3,000 µV hundreds of times per second. Directly driving a VR 3D avatar with raw rectified samples would cause the virtual arm to violently vibrate and jitter.

To transform rectified spikes into a smooth, steady, physically meaningful muscle force curve, biomedical engineering employs the **Root Mean Square (RMS) Envelope Detector**:

The Universal Mathematical Definition of Moving-Window Root Mean Square

$$\text{RMS}[n] = \sqrt{ \frac{1}{N} \sum_{k=0}^{N-1} y^2[n-k] }$$

Deconstructing the Three Operations of RMS:

The name "Root Mean Square" describes its exact step-by-step computational recipe, executed in reverse order:

SQUARE: y^2[n-k]: Every individual sample in the observation window is squared. Squaring achieves two vital goals: (a) It guarantees all numbers are positive, since the square of any real number is positive. (b) It acts as a non-linear amplifier that emphasizes high-amplitude action potential bursts while suppressing low-level resting noise floor.

MEAN: (1/N) Σ (...): All N squared values across the observation window are summed together and divided by N. This computes the mathematical average of the squared signal, which represents the **temporal electrical power density** of the active muscle!

ROOT: √(...): Taking the square root of the mean power reverses the squaring operation, bringing the physical units back from squared microvolts (μV^2) to linear microvolts (μV), matching the BioRadio's calibration scale.

🔬 **Technical Fact & Definition: Parseval's Theorem and Physical Muscle Power**

Why is RMS universally preferred over Mean Absolute Value (MAV = Σ|y|/N) in quantitative biomechanics?

The theoretical justification rests on **Parseval's Theorem** (Lathi Section 7.4):

$$E = \int |x(t)|^2 \, dt = (1 / 2\pi) \int |X(j\omega)|^2 \, d\omega$$

The integral of a signal squared across time equals the total integral of its energy across all frequencies! Therefore, the **Mean Square of an EMG signal is a direct mathematical measurement of total electrical energy dissipation** in the contracting muscle fibers. In muscle physiology, total electrical energy is directly correlated with the rate of ATP hydrolysis and actin-myosin cross-bridge mechanical cycling!

9.3 Moving-Window RMS Buffer Implementation

Let us examine the exact production Python code written in `emg_methods.py` for the `Streaming.envelope()` method:

emg_methods.py: Production Moving-Window RMS Envelope Algorithm

```
 # Circular history buffer initialized in __init__:
# Maintains rolling buffer of recent processed samples (500 ms history at fs=1000 Hz)
self.history = deque(maxlen=int(0.5 * self.fs))

# Envelope Method in Streaming Class:
def envelope(self, window_s=0.100):
    """
    Compute causal moving-window Root Mean Square (RMS) envelope.
    :param window_s: Duration of observation window in seconds (default = 100 ms)
    :return: Instantaneous muscle power in microvolts (uV)
    """
    # Step 1: Calculate number of discrete samples in window
    n = max(1, int(window_s * self.fs))
    # Step 2: Convert circular deque into contiguous float numpy array
    h = np.fromiter(self.history, dtype=float)
    if h.size == 0:
        return 0.0
    # Step 3: Extract the most recent n samples, square them, take mean, and square root
    return float(np.sqrt(np.mean(h[-n:] ** 2)))
```

Algorithmic Nuances:

- `deque(maxlen=int(0.5 * self.fs))`: At f_s = 1000 Hz, this double-ended queue stores exactly 500 samples (0.500 seconds of history). As new samples enter, the oldest samples automatically pop off in O(1) constant time.
- `n = max(1, int(window_s * self.fs))`: For a standard `window_s = 0.100` (100 ms), n = 100 samples. The `max(1, ...)` safeguard prevents division-by-zero errors.
- `np.mean(h[-n:] ** 2)`: Slices the tail of the array (the 100 most recent samples), squares each element via SIMD vectorization, and computes the arithmetic mean.
- `np.sqrt(...)`: Computes the scalar square root, returning the instantaneous RMS amplitude in microvolts.

9.4 The Linear Envelope Curve

In clinical biomechanics and kinesiology, the curve resulting from low-pass filtering or moving-RMS averaging of rectified EMG is formally known as the **Linear Envelope**.

When an athlete relaxes, the linear envelope sits at a quiet baseline of 10 to 30 µV. When the athlete initiates a curl, the envelope rises smoothly into a rounded, continuous hill that peaks at 1,500 to 4,000 µV during peak contraction, then gently glides back down as the muscle relaxes.

This smooth hill mirrors the macroscopic mechanical tension generated by the muscle tendon pulling on the bone, providing the ideal continuous control input for virtual weightlifting.

9.5 Window Size Selection and Latency Trade-offs

Selecting the observation window duration (T_w = window_s) is a classic engineering optimization problem balancing **statistical variance (ripple)** against **temporal responsiveness (latency)**:

Window Duration (T_w)	Samples (at 1 kHz)	Envelope Ripple / Noise	Effective Group Delay	User Feel & Game Impact
25 ms (Short)	25 samples	High ripple; jagged, jumpy peaks	12.5 ms (Ultra-fast)	Avatars twitch nervously; feels jittery and unstable
100 ms (Medium) [Selected]	100 samples	Low ripple; smooth, stable curve	**50.0 ms (Optimal)**	**The Golden Standard**: smooth motion with zero perceptible control lag
200 ms (Long)	200 samples	Zero ripple; perfectly flat curve	100.0 ms (Sluggish)	Noticeable sluggishness; avatar feels like it is moving through molasses

The group delay of a moving-window RMS filter is exactly half its window duration:

Group Delay of a Rectangular Moving-Window RMS Integrator

$$\tau_{RMS} = \frac{T_w}{2} = \frac{100\,\text{ms}}{2} = 50.0\,\text{ms}$$

By combining a 100 ms RMS window with our high-speed filters, total signal processing latency remains safely below 100 ms, delivering instantaneous cybernetic responsiveness!

PART IV: FROM FILTERED SIGNAL TO MUSCLE POWER NUMBER • CHAPTER 10

Two-Point Calibration — Solving the 6 kg Weightlifting Mystery

Diagnosing the isometric contraction paradox, percentile-based outlier rejection, dead-zone transformations, and frame-rate-independent exponential smoothing.

10.1 The 6 kg Dumbbell Paradox: The Mystery that Baffled the Lab

During the third week of experimental trials at IIT Gandhinagar, the student research team encountered an exasperating anomaly that initially seemed like a total software failure:

When an athlete tested the system without weights (the "No Load" condition), the VR weightlifting simulator functioned flawlessly. The athlete stood with their arm relaxed; the virtual avatar's arm hung straight down. The athlete curled their arm; the virtual avatar curled synchronously.

However, the moment the athlete picked up a physical **6 kg iron dumbbell** and stood completely motionless with their arm hanging straight down, **the virtual avatar's arm immediately flexed and lifted the dumbbell to its chest**, even though the athlete had not bent their elbow by a single millimeter!

> ⚠️ **Critical Pitfall & Warning: The Naive DSP Trap**
>
> The team's initial instinct was that the digital filters were failing. Perhaps 50 Hz power-line noise was leaking into the circuit? Raima tightened the notch filter. Kunal increased FIR taps to 201. Chandan narrowed the FFT mask. Krishna sharpened the differentiator.
>
> **None of it worked! The virtual arm still curled up on its own!**
>
> Why? Because the signal coming from the electrodes was NOT electrical noise. It was a 100% genuine, pristine, physiological muscle action potential generated by the human body!

10.2 Biomechanics of Isometric Contraction & Co-Contraction

To solve this mystery, the team had to look away from circuit diagrams and study functional human biomechanics.

When you hold a 6 kg dumbbell with your arm hanging vertically downward, gravity exerts a downward tensile force of approximately $F = m \cdot g = 6\ \text{kg} \times 9.81\ \text{m/s}^2 \approx 58.9$ Newtons on the forearm. If your muscles were completely turned off (zero electrical activity), this 59 N downward pull would stretch your tendons and dislocate the delicate glenohumeral and humeroulnar joint capsules!

To protect the elbow and shoulder joints from traction injury, the central nervous system automatically engages in **Tonic Isometric Contraction and Co-Contraction**:

1. **Joint Traction Stabilization:** The biceps brachii and brachioradialis fire tonic, low-frequency motor units to maintain tension across the elbow joint, resisting gravitational distraction.
2. **Grip Synergistic Irradiation:** Holding a heavy 6 kg metal handle requires gripping it tightly. Forearm flexor motor units discharge intensely. Due to neural irradiation in the spinal cord, high-force gripping automatically recruits synergistic stabilizing contractions in the upper arm biceps!
3. **The Physiological Voltage Surge:** At true rest (0 kg load), the biceps envelope sits at 15 to 30 µV. When simply holding the 6 kg dumbbell with arm straight, the biceps envelope rises to **150 to 350 µV**!
4. **The Software Error:** The naive software had assumed that any voltage above 50 µV represented an active curl. Because 250 µV exceeded the 50 µV threshold, the virtual avatar faithfully curled the arm!

> 🔬 **Technical Fact & Definition: The Critical Engineering Insight**
>
> **Digital filters cannot distinguish between 250 µV generated by holding a heavy weight and 250 µV generated by flexing an unloaded arm!**
>
> Both signals have identical frequencies (20–450 Hz) and identical wave shapes. The solution cannot be found in frequency-domain filtering. The solution demands **Adaptive Context-Aware Calibration**!

10.3 The Two-Point Calibration Algorithm

To conquer the 6 kg mystery, the team engineered a robust, automated **Two-Point Calibration System**. Instead of hardcoding arbitrary microvolt thresholds, the system dynamically measures two physiological reference points for the specific athlete under their specific load condition:

Point 1: REST Calibration — 5.0 Seconds Duration

The athlete holds the dumbbell motionless with their arm hanging straight down for 5.0 seconds. The system records all RMS envelope samples and extracts the **95th percentile** as `rest_level`.

Point 2: MAX Calibration — 8.0 Seconds Duration

The athlete performs 3 complete, maximal bicep curls with the dumbbell over an 8.0-second window. The system records all RMS envelope samples and extracts the **99th percentile** as `max_level`.

Why Percentiles Instead of Simple Min/Max or Mean?

Why did the team specifically select the **95th and 99th percentiles** instead of taking the simple average or mathematical maximum?

> 📖 **B.P. Lathi Textbook Reference (Statistical Robustness): Outlier Immunity via Percentile Ranking**
>
> In real-world biomedical recording, biological signals are plagued by sporadic non-Gaussian outliers:
>
> 1. **Why NOT the Mean for REST?** If an athlete swallows, coughs, or shifts their weight during the 5-second rest test, a single 100 ms burst will pull the mathematical mean upward, resulting in a false rest baseline.
>
> 2. **Why NOT the Maximum for REST?** If an electrode adhesive patch pops slightly, it generates a single 1,000 µV spike. If the system took the maximum, `rest_level` would be set to 1,000 µV, requiring the athlete to produce impossible voltages just to register a curl!
>
> 3. **The 95th Percentile Solution:** Sorting all 250 envelope samples recorded over 5 seconds and taking the 95th percentile captures the true upper ceiling of continuous resting tone while effortlessly discarding the top 5% of accidental coughs and twitch outliers!
>
> 4. **The 99th Percentile for MAX:** Similarly, taking the 99th percentile of the 8-second curl test captures the sustained plateau of maximal voluntary contraction while ignoring single-sample artifact spikes.

10.4 The Normalization & Clamping Equation

With `rest_level` and `max_level` firmly established, the raw microvolt envelope (env) is mapped into a normalized physical activation metric (a) bounded strictly between 0.0 (complete rest) and 1.0 (maximal voluntary curl):

The Linear Normalization Mapping Equation

$$a = \frac{env - \text{rest_level}}{\max(\text{max_level} - \text{rest_level}, 10^{-6})}$$

The denominator protection term max(..., 10^{-6}) guarantees that even if a confused user performs calibration incorrectly (where max_level ≈ rest_level), the software will never crash with a division-by-zero floating-point exception.

The raw activation is then hard-clamped to the unit interval [0.0, 1.0]:

Bounding Activation Between 0% and 100%

$$a_{clamped} = \min(1.0, \max(0.0, a))$$

10.5 The 5% Dead-Zone Transformation

Even after percentile calibration, human muscles are not rigid metal statues. While an athlete holds the 6 kg weight at rest, microscopic variations in recruitment cause the envelope to occasionally hover slightly above `rest_level`, yielding normalized activation values fluctuating between 0.01 and 0.04.

If this tiny baseline noise is fed directly into the game, the VR avatar's hand will exhibit an annoying, nervous tremor at rest.

To eliminate resting jitter, the bridge applies a **Non-Linear 5% Dead-Zone Transformation** (parameterized by `DEAD_ZONE = 0.05`):

Piecewise Mathematical Definition of the 5% Dead-Zone Function

$$a_{final} = \begin{cases} 0.0 & \text{if } a_{clamped} < 0.05 \\[6pt] \displaystyle\frac{a_{clamped} - 0.05}{1.0 - 0.05} & \text{if } a_{clamped} \ge 0.05 \end{cases}$$

Notice the denominator (1.0 − 0.05) = 0.95: this rescale ensures that as $a_{clamped}$ rises from 0.05 to 1.00, the final output a_{final} scales smoothly and linearly from 0.00 to 1.00 without a sudden jump or discontinuity!

10.6 Continuous Exponential Smoothing

The final processing step before telemetry transmission is **Continuous Exponential Smoothing**. In discrete time, this is implemented as:

Frame-Rate-Independent Continuous Exponential Smoothing Equation

$$\text{act_smooth} \mathrel{+}= \left(1 - e^{-\Delta t / \tau} \right) \cdot (a_{final} - \text{act_smooth})$$

Where Δt is the elapsed time since the previous update (in seconds) and τ is the filter time constant.

🔬 **Technical Fact & Definition: Equivalence to an Analog 1st-Order Low-Pass Filter**

The exponential smoothing equation is mathematically identical to a continuous-time 1st-order RC low-pass filter with transfer function:

$$H(s) = 1 / (1 + s\tau)$$

Using the exponential decay factor `(1 - exp(-dt / τ))` guarantees that the smoothing behavior is **100% frame-rate independent**! Whether the Python loop updates at 50 Hz, 45 Hz, or 60 Hz, the physical step response of the smoothing filter remains perfectly constant.

Smoothing Preset	Time Constant (τ)	Visual Feel in Unity VR	Optimal Application
'Off'	τ = 0.000 s	Raw, instantaneous, immediate feedback; slight mechanical twitch visible	Rapid reflex testing, reaction-time benchmarking
'100 ms' [Default]	τ = 0.100 s	**Balanced, natural biomechanical feel**; eliminates jitters while maintaining sub-100ms response	**Primary VR Weightlifting Mode**: feels authentic and responsive
'200 ms'	τ = 0.200 s	Buttery smooth, heavy cinematic movement; slight perceived lag	Exhibition demos, cinematic replays, high-tremor subjects

PART V: REAL-TIME COMMUNICATION AND DATA LOGGING • CHAPTER 11

UDP Network Telemetry & Protocol Architecture

Architecting a zero-latency cybernetic bridge: evaluating TCP versus UDP transport semantics, localhost socket programming, JSON telemetry serialization, and dropped-packet resilience.

11.1 The Transport Layer: TCP vs UDP for Real-Time Biofeedback

Once Python has filtered the raw microvolts, computed the RMS envelope, calibrated the baseline, and smoothed the activation metric, this data must travel across process boundaries into the **Unity 3D VR Engine** to animate the virtual athlete's arm.

In the OSI 7-layer telecommunications model, communication between two software applications running concurrently on the same computer is governed by the **Transport Layer (Layer 4)**. An engineer faces an immediate architectural fork in the road: should we use the **Transmission Control Protocol (TCP)** or the **User Datagram Protocol (UDP)**?

> 💡 **Intuitive Analogy: The Registered Letter vs The Live Radio Broadcast**
>
> Imagine you are listening to a live radio sports broadcast of a fast-paced basketball game. The commentator shouts: "He shoots... it's in!" (a real-time biofeedback packet).
>
> Now imagine if radio worked like **TCP**: if your radio suffered a tiny burst of static and missed the word "shoots", the broadcast would completely freeze! The radio station would pause the entire game, look up the missing audio, ship it to your house in a registered envelope, wait for you to sign an acknowledgment receipt, and only then resume the broadcast—5 minutes late!
>
> By the time you hear "He shoots", the entire game is over! That is **TCP Head-of-Line Blocking**.
>
> **UDP is like a live FM radio broadcast**: the radio station streams sound continuously into the air. If one syllable gets clipped by static, you don't stop the broadcast; you simply listen to the next syllable that arrives 20 milliseconds later! For real-time VR muscle control, UDP is the only acceptable choice.

Protocol Characteristic	Transmission Control Protocol (TCP)	User Datagram Protocol (UDP) [Selected]
Connection Model	Connection-oriented (3-way SYN-ACK handshake)	Connectionless ("Fire-and-Forget" datagrams)
Packet Delivery Guarantee	Guaranteed delivery via ACK packets and retransmissions	Unreliable; packets may be dropped without notification
Ordering Guarantee	Strict in-order sequence numbers; missing packets stall queue	No ordering guarantee; packets processed as they arrive
Head-of-Line Blocking	**SEVERE**: A single dropped packet stalls all newer data	**ZERO**: Dropped packets are ignored; fresh data flows freely
Packet Overhead	Heavy (20 to 60 bytes header + flow control state)	Ultra-lightweight (exactly 8 bytes header)
Suitability for Biofeedback	Terrible for VR; induces stutter, lag, and motion sickness	**Ideal**: sub-millisecond local transit with fresh data at 50 Hz

11.2 Localhost Socket Programming in Python

To establish this high-speed inter-process communication (IPC) channel, the Python bridge initializes a standard BSD datagram socket bound to the **IPv4 Loopback Interface**:

bioradio_unity_bridge.py: UDP Datagram Socket Initialization

```
import socket
import json

# Network Constants defined in bioradio_unity_bridge.py
UDP_HOST = '127.0.0.1'   # Standard IPv4 Localhost Loopback Interface
UDP_PORT = 9050          # Safe, unblocked UDP destination port for Unity (> 8080)

# Initialize UDP IPv4 Datagram Socket
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
# Set socket to non-blocking mode to prevent GUI thread stalls
sock.setblocking(False)
```

Why the Localhost Loopback Interface (127.0.0.1)?

When packets are addressed to `127.0.0.1`, the operating system's networking stack performs a short-circuit optimization:

- The packet is routed entirely inside kernel memory space (RAM).
- It never touches a physical network interface card (NIC), Ethernet cable, or Wi-Fi radio.
- It suffers zero electromagnetic interference, zero physical collision, and zero transmission delays.
- The transit time from Python `sock.sendto()` to the Unity C# receiver buffer is **less than 0.2 milliseconds (200 microseconds)**!

11.3 Telemetry JSON Payload Schema

Every 20 milliseconds, the Python bridge packages all current biosignal telemetry into a structured **JavaScript Object Notation (JSON)** string encoded into UTF-8 binary bytes:

Python Telemetry Serialization and UDP Dispatch

```
# Telemetry payload dispatched every 20 ms:
payload = {
    "timestamp": round(time.time(), 4),          # Wall-clock timestamp (seconds)
    "method": self.active_method,                 # Active DSP filter ('IIR', 'FIR', 'FFT', 'DIFF')
    "raw_uV": round(float(raw_sample), 2),        # Instantaneous raw electrode potential (uV)
    "filtered_uV": round(float(filt_sample), 2),  # Instantaneous filtered potential (uV)
    "envelope_uV": round(float(env_uV), 2),       # Moving-window RMS envelope (uV)
    "activation": round(float(act_smooth), 4),    # Normalized, dead-zoned activation [0.0 to 1.0]
    "rest_level": round(float(self.cal_rest), 2), # Calibrated rest baseline (uV)
    "max_level": round(float(self.cal_max), 2),   # Calibrated maximal curl ceiling (uV)
    "load_state": self.load_condition             # Current load metadata ('No load', '6 kg')
}
raw_bytes = json.dumps(payload).encode('utf-8')
sock.sendto(raw_bytes, (UDP_HOST, UDP_PORT))
```

Comprehensive JSON Schema Field Reference:

Field Key	Data Type	Physical Units	Description & Role in Unity VR
`timestamp`	Float	Seconds (s)	High-resolution wall-clock timestamp used by Unity to compute latency and frame synchronization.
`method`	String	Categorical	Identifies which student's filter is currently driving the avatar (`'IIR'` , `'FIR'` , `'FFT'` , `'DIFF'`).
`raw_uV`	Float	Microvolts (μV)	Unfiltered biosignal amplitude used to render real-time oscilloscopes inside the VR laboratory HUD.
`filtered_uV`	Float	Microvolts (μV)	Purified bandpass/notched biosignal showing the clean biological MUAP spikes.
`envelope_uV`	Float	Microvolts (μV)	The 100 ms RMS power envelope tracking continuous muscle effort.
`activation`	Float	Normalized [0, 1]	**The Master Control Value**: mapped directly to the elbow joint angle rotation of the 3D VR character.
`rest_level`	Float	Microvolts (μV)	The 95th percentile baseline ceiling established during the 5-second rest calibration.
`max_level`	Float	Microvolts (μV)	The 99th percentile maximum contraction ceiling established during the 8-second curl calibration.
`load_state`	String	Categorical	Tracks whether the user is curling with `'No load'` or the `'6 kg'` dumbbell for game score multipliers.

11.4 Update Frequency and Packet Drop Resilience

The telemetry pipeline operates at a target dispatch rate governed by:

Telemetry Broadcast Frequency (50 Hz Update Loop)

$$f_{update} = \frac{1000\,\text{ms}}{\text{UPDATE_MS}} = \frac{1000\,\text{ms}}{20\,\text{ms}} = 50.0\,\text{Messages / Second}$$

At 50 Hz, a new telemetry packet arrives every 20.0 milliseconds. This timing integrates seamlessly with modern VR headsets:

- Standard VR head-mounted displays (such as the Meta Quest 3, HTC Vive Pro, or Valve Index) refresh at **72 Hz, 90 Hz, or 120 Hz** (one visual frame every 13.8 ms, 11.1 ms, or 8.3 ms).
- A 50 Hz telemetry stream guarantees that Unity receives a fresh muscle activation update every 1 to 2 visual frames.

What Happens If a Packet is Dropped?

In the rare event that an operating system context-switch causes a UDP buffer overrun, what is the consequence?

Absolutely nothing! Because each UDP datagram is completely self-contained and carries the absolute, instantaneous muscle activation value (NOT an incremental delta), Unity simply continues rendering the previous frame. 20 milliseconds later, the next packet arrives containing the current physical reality. The user never notices a drop, ensuring butter-smooth VR weightlifting without stutters or hitching!

11.5 Systems Engineering Case Study: The Windows Hyper-V Port Exclusion Bug

During hardware validation on Windows 11 development machines, our team encountered an instructive real-world operating systems anomaly that provides vital architectural insight for future biomedical and VR cybernetic deployments.

Initially, the telemetry channel was configured to transmit on high-range ephemeral port `50557` . However, while Python reported successful transmission, Unity's C# socket listener threw the following fatal socket exception upon startup:

Unity Player.log Runtime Socket Failure

```
[UDPReceiver] Error binding to port 50557:
An attempt was made to access a socket in a way forbidden by its access permissions.
System.Net.Sockets.SocketException (10013): Access is denied.
```

Diagnostic Root Cause Analysis

Auditing the Windows TCP/IP networking stack via the command line revealed the culprit:

Windows Kernel Dynamic Port Exclusion Ranges

```
netsh interface ipv4 show excludedportrange protocol=udp

Protocol udp Port Exclusion Ranges
Start Port    End Port
----------    --------
     50000       50059     *
     50540       50639       <-- Port 50557 trapped in kernel exclusion range!
     52110       52209
     52510       52609
* - Administered port exclusions.
```

Windows Hyper-V and the Host Network Service (HNS) dynamically reserve large contiguous blocks of high-range UDP ports for containerized networking and virtualization subsystems. When an application attempts to execute a POSIX `bind()` on any port inside an excluded range, the Windows kernel rejects the socket with error code **WSAEACCES (10013)**.

Three-Pillar Architectural Resolution:

Migrated to Port 9050: Port 9050 is well above 8080 and safely outside all Windows dynamic kernel virtualization exclusion bands (which reside predominantly above 50,000). Direct loopback socket benchmarks verified sub-millisecond binding with zero permission conflicts.

0.0.0.0 with 127.0.0.1 Fallback: UDPReceiver.cs was re-architected with defensive dual-bind logic: it attempts to bind to IPAddress.Any (0.0.0.0), and if restricted by interface policies, automatically falls back to IPAddress.Loopback (127.0.0.1).

Application.runInBackground = True: By default, the Unity game loop pauses when the application window loses desktop focus. Setting Application.runInBackground = true guarantees that UDP reception and avatar skeletal physics continue processing without interruption while the user operates the BioRadio Python bridge window.

PART V: REAL-TIME COMMUNICATION AND DATA LOGGING • CHAPTER 12

The Python Bridge Application Architecture

Designing a multi-threaded biosignal acquisition bridge: C# CLR interop via pythonnet, simultaneous parallel filter execution, real-time Matplotlib oscilloscopes, and scientific CSV telemetry logging.

12.1 High-Level Architecture Overview

The central nervous system of our physical-digital cybernetic installation is the **Python Bridge Application** (implemented in `bioradio_unity_bridge.py`). This application acts as a high-performance intermediary, choreographing hardware driver access, scientific digital signal processing, user interface rendering, and network socket telemetry.

- **Hardware Driver Ingestion:** Polls the GLNT BioRadio hardware via unmanaged C# .NET dynamic link libraries, extracting 16-bit digitized voltage samples at 1,000 Hz.
- **Parallel DSP Filtering:** Feeds raw microvolt arrays into four concurrent digital filter instances (IIR, FIR, FFT, DIFF) to maintain continuous internal delay memory.
- **Envelope Detection & Calibration:** Calculates the 100 ms moving-window RMS envelope, applies load-dependent percentile normalization, and evaluates non-linear dead-zones.
- **Network Telemetry Dispatch:** Serializes telemetry into JSON packets dispatched over UDP socket 9050 to Unity at 50 Hz.
- **GUI Rendering & CSV Logging:** Updates real-time Matplotlib oscilloscopes at 50 FPS and appends full 12-column scientific telemetry records to disk for post-hoc research validation.

12.2 Interfacing BioRadioSDK.dll via Pythonnet (CLR Interop)

The vendor hardware manufacturer (Great Lakes NeuroTechnologies) provides device drivers compiled as unmanaged C# .NET assemblies. To access these low-level drivers directly from a standard CPython 3.10 interpreter without writing fragile C++ wrapper layers, the team employed **Python for .NET (`pythonnet`)**:

Loading Unmanaged C# .NET BioRadio Drivers via Pythonnet

```
import clr
import sys
import os

# 1. Add directory containing BioRadioSDK.dll to Python DLL search path
dll_path = os.path.abspath(r".\\BioRadio_Student_Unity_Kit")
sys.path.append(dll_path)

# 2. Bind .NET Common Language Runtime (CLR) to unmanaged DLL
clr.AddReference("BioRadioSDK")
import BioRadioSDK

# 3. Instantiate and configure hardware device object
device = BioRadioSDK.BioRadio()
device.FindBioRadio()
device.StartAcquisition()
```

This CLR bridge permits Python to communicate directly with the underlying Windows USB/Bluetooth driver stack, querying FIFO ring buffers and pulling digitized integer sample packets with zero copy overhead.

12.3 The Parallel Processing Engine: Simultaneous Multi-Filter Execution

A cornerstone design innovation of SSRP Project 7 is the **Parallel Multi-Method Engine**.

In naive software, when a user selects a filter from a dropdown menu (e.g., switching from Raima's IIR to Kunal's FIR), the application initializes the new filter on the fly. As established in Chapters 5 and 6, uninitialized digital filters suffer violent transient pops because their delay registers start cold!

To enable instantaneous, glitch-free hot-swapping during live gameplay, the Python bridge initializes **all four filters concurrently at startup**:

Parallel Execution of All Four Filters Inside the 20 ms Update Loop

```
 # Initializing all 4 filter superheroes concurrently in memory
METHODS = ['IIR', 'FIR', 'FFT', 'DIFF']
OWNER = {'IIR': 'Raima', 'FIR': 'Kunal', 'FFT': 'Chandan', 'DIFF': 'Krishna'}

# Dictionary containing 4 dedicated Streaming instances
self.streamers = {m: Streaming(method=m, fs=1000.0) for m in METHODS}

# Inside the main 20 ms update loop:
for m in METHODS:
    # All 4 filters process the identical incoming chunk simultaneously!
    # This keeps internal states (zi_bp, fir_zi, fft_buf, diff_prev) perfectly warmed up!
    y_filtered = self.streamers[m].process(raw_chunk)
    self.latest_filtered[m] = y_filtered[-1]
    self.latest_envelope[m] = self.streamers[m].envelope(window_s=0.100)
```

Because all four filters process every incoming chunk in parallel, their internal state registers (z_i) and circular history queues remain continuously warmed up and synchronized with real physical time. When the athlete clicks a radio button in the GUI during a heavy curl, the system switches the active control method in **zero microseconds with absolute mathematical smoothness**!

12.4 Real-Time GUI Visualization with Matplotlib & Tkinter

The user interface is constructed using standard Python `tkinter` with embedded `matplotlib` canvas plotting:

- **Non-Blocking Event Loop:** The application avoids blocking `while True: time.sleep()` loops. Instead, it leverages Tkinter's asynchronous timer: `root.after(UPDATE_MS, self.update_loop)`. This guarantees that GUI buttons remain snappy and responsive.
- **Dual-Trace Oscilloscope:** The upper oscilloscope renders the high-frequency raw microvolt potential (capturing bi-phasic MUAP bursts), while the lower oscilloscope renders the rectified 100 ms RMS envelope (showing the smooth activation hill).
- **Blit Canvas Optimization:** Rather than redrawing entire chart axes and tick labels on every frame, Matplotlib blitting updates only the pixel bounding boxes of the data lines, maintaining a rock-solid 50 FPS refresh rate on consumer hardware.

12.5 Scientific CSV Logging: 12-Column Telemetry Specification

To ensure absolute academic rigor, reproducibility, and post-hoc statistical validation for the SSRP 2026 symposium, the bridge records an exhaustive 12-column telemetry dataset saved to a timestamped CSV file:

Column Index	Header Name	Data Format	Scientific Description & Analysis Purpose
1	`Timestamp`	Float (seconds.ms)	High-resolution monotonic system time since recording onset.
2	`Sample_Index`	Integer	Sequential integer counter tracking total samples processed.
3	`Raw_uV`	Float (µV)	Unprocessed biosignal directly from BioRadio ADC.
4	`IIR_Filt_uV`	Float (µV)	Raima's 4th-order Butterworth SOS bandpass + notch filtered potential.
5	`FIR_Filt_uV`	Float (µV)	Kunal's 101-tap linear-phase Hamming FIR filtered potential.
6	`FFT_Filt_uV`	Float (µV)	Chandan's 256-point block spectral masked filtered potential.
7	`DIFF_Filt_uV`	Float (µV)	Krishna's 1st-order numerical difference + notch filtered potential.
8	`RMS_Envelope_uV`	Float (µV)	Instantaneous 100 ms moving-window Root Mean Square envelope.
9	`Norm_Activation`	Float [0.0 – 1.0]	Final smoothed, dead-zoned activation driving the VR avatar.
10	`Load_Condition`	String	Experimental metadata tag (`'No load'` or `'6 kg'`).
11	`Cal_Rest_uV`	Float (µV)	The 95th percentile baseline ceiling used for normalization.
12	`Cal_Max_uV`	Float (µV)	The 99th percentile maximum contraction ceiling used for normalization.

Because all four filtered outputs (Columns 4, 5, 6, and 7) are logged side-by-side for the exact same physical contraction, researchers can perform post-hoc statistical analyses (calculating Signal-to-Noise Ratio, spectral power distortion, and correlation coefficients) across all four student methods on identical biological data!

PART V: REAL-TIME COMMUNICATION AND DATA LOGGING • CHAPTER 13

The Complete Signal Processing Pipeline & Latency Budget

Synthesizing the complete end-to-end cybernetic loop from motor cortex depolarization to 3D virtual reality animation, itemizing the cumulative latency budget, and delivering the master scorecard of all four filtering methods.

13.1 The End-to-End Cybernetic Journey: Brain to Virtual Reality

To conclude our master reference guide, we trace the complete, unbroken physical, biological, mathematical, and computational path traveled by a human thought as it transforms into a virtual reality action:

Primary Motor Cortex: Pyramidal neurons in the motor cortex generate electrical action potentials encoding the conscious intention to curl a dumbbell.

Alpha Motor Neuron: Impulses travel down the corticospinal tract and synapse with α-motor neurons in the spinal cord, broadcasting signals down peripheral nerves to the biceps.

Neuromuscular Junction: Acetylcholine triggers sodium influx at the motor endplate. Depolarization waves propagate along thousands of muscle fibers at 3–5 m/s.

Biological Tissue: Muscle fiber currents summate into Motor Unit Action Potentials (MUAPs), propagating through interstitial fluid, fat, and skin to the surface.

Ag/AgCl Electrodes: Silver/silver chloride electrodes with conductive chloride gel translate ionic currents into electronic currents via reversible oxidation-reduction reactions.

Instrumentation Amplifier: A 3-op-amp instrumentation amplifier provides >110 dB CMRR, subtracting common-mode environmental hum and amplifying the differential signal by 1,000×.

16-Bit Delta-Sigma ADC: The amplified voltage is sampled at exactly 1,000.0 Hz (1 sample every 1.0 ms) with 0.3 µV quantization resolution, satisfying the Nyquist theorem.

Bluetooth 2.1 RFCOMM: Digitized samples are packetized and streamed wirelessly over Bluetooth to the host computer's virtual COM port.

Python Signal Engine: Samples enter `emg_methods.py`, passing through Raima's IIR, Kunal's FIR, Chandan's FFT, or Krishna's DIFF filter to destroy motion artifacts and 50 Hz hum.

Moving-Window RMS: A 100 ms causal sliding window computes the Root Mean Square power, converting alternating bipolar spikes into a smooth, unipolar linear envelope in microvolts.

Cybernetic Normalization: The envelope is normalized against load-dependent 95th/99th percentiles, evaluated against a 5% dead zone, and exponentially smoothed (τ = 100 ms).

Virtual Reality Simulation: The smoothed activation [0.0 to 1.0] is serialized into a JSON packet and dispatched via UDP socket 9050 to Unity, which rotates the avatar's elbow joint in VR!

13.2 Cumulative Latency Budget Analysis

In virtual reality and cybernetics, **Latency is the Ultimate Enemy**. If the delay between a physical muscle contraction and the corresponding movement in the VR headset exceeds 100 to 150 milliseconds, the human brain detects the dissociation. The user experiences control frustration, immersion breaks, and vestibulo-ocular disorientation (cybersickness).

To ensure complete transparency, we itemize the cumulative latency contributed by every single stage of the SSRP Project 7 pipeline:

Pipeline Subsystem	Physical Mechanism	Latency Contribution (Typical)	Cumulative Delay
1. Biological Conduction	Neuromuscular transmission & sarcolemma propagation	≈ 5.0 ms – 10.0 ms	10.0 ms
2. Hardware ADC & Analog Filter	BioRadio analog anti-aliasing filter & delta-sigma conversion	≈ 1.5 ms – 2.0 ms	12.0 ms
3. Bluetooth Transmission	Firmware packetization, RFCOMM serial buffer, and OS driver	≈ 10.0 ms – 15.0 ms	25.0 ms
4. Python Chunk Ingestion	20-sample batch accumulation (20 ms chunk interval / 2)	≈ 10.0 ms (Average)	35.0 ms
5. DSP Filter Group Delay	Method Dependent: • Raima (IIR): 8 ms passband + notch = **~18 ms** • Kunal (FIR): Constant = **50.0 ms** • Chandan (FFT): Buffer centroid = **~128.0 ms** • Krishna (DIFF): Differentiator = **0.5 ms**	• IIR: 18 ms • FIR: 50 ms • FFT: 128 ms • DIFF: 0.5 ms	IIR: 53 ms FIR: 85 ms FFT: 163 ms DIFF: 35.5 ms
6. Moving-Window RMS (100 ms)	Rectangular window group delay ($\tau = T_w / 2$)	≈ 50.0 ms	IIR: ~60–75 ms FIR: ~85–100 ms
7. UDP Localhost Network Transit	Kernel RAM loopback socket (127.0.0.1:9050)	< 0.2 ms	+0.2 ms
8. Unity VR Render Frame	Game loop update & 90 Hz VR display scanout (1 frame)	≈ 11.1 ms	**Total Perceived System Latency: ~70 to 95 ms**

> 🔬 **Technical Fact & Definition: The 100 ms Cybernetic Victory**
>
> The total end-to-end perceived latency of the SSRP Project 7 system—measured from the instant alpha motor neurons fire to the instant the VR avatar's virtual dumbbell begins moving—is **between 70 ms and 95 ms** (using Raima's IIR or Kunal's FIR filter)!
>
> Because human voluntary reaction time to visual stimuli is approximately 180 to 220 milliseconds, a system response of 80 milliseconds feels **completely instantaneous, fluid, and natural** to the user. The avatar moves as if connected directly to the user's mind!

13.3 Grand Comparative Scorecard of the Four Methods

To synthesize the semester's research at IIT Gandhinagar, we present the definitive master scorecard comparing the four filter implementations across all engineering, mathematical, and pedagogical criteria:

Evaluation Metric	Raima Ghosh (Butterworth IIR)	Kunal Kashyap (Hamming FIR)	Chandan Kumar (FFT Spectral Mask)	Krishna Patel (Difference Operator)
Mathematical Architecture	Recursive Feedback Difference Eq. (4-stage Second-Order Sections)	Non-Recursive Convolution (101-tap linear-phase)	Discrete Fourier Transform (256-pt Cooley-Tukey Radix-2)	First-Order Backward Difference ($y[n] = x[n] - x[n-1]$)
Theoretical Stability	Conditionally stable (Poles bounded inside unit circle via SOS)	**Unconditionally Stable** (All 100 poles at origin)	Inherently stable (Direct block transform)	**Unconditionally Stable** (Single pole at origin)
Phase Linearity	Non-linear phase (Slight high-frequency delay shift)	**Strict Linear Phase** (Constant 50 ms group delay)	Linear within masked passband (Edge phase discontinuities)	Linear Phase ($\theta(\Omega) = \pi/2 - \Omega/2$)
Waveform Fidelity	Good (Minor biological spike smearing)	**Superb** (100% MUAP wave preservation)	Moderate (Sinc Gibbs ringing near mask transitions)	Moderate (Distorts shape via differentiation)
Motion Artifact (<20 Hz) Rejection	**Exceptional** (−80 dB/decade slope)	**Exceptional** (>50 dB attenuation)	**Surgical** (Bins 0–4 set to 0.0 + 0.0j)	Good (Zero at DC, slopes +20 dB/decade)
50 Hz Mains Hum Rejection	**Razor-Sharp** (IIR Notch, Q=30, BW ≈ 1.67 Hz)	**Razor-Sharp** (Cascaded IIR Notch, Q=30)	**Surgical** (±2 Hz bin mask zeroing)	**Razor-Sharp** (Cascaded IIR Notch, Q=30)
CPU Cost per Sample	**Ultra-Low** (16 multiplies, 16 adds)	Moderate (101 multiplies, 100 adds)	High (O(N log N) FFT every 20 ms)	**Minimalist** (1 subtraction per sample)
Group Delay (Filter Only)	Low (≈ 18 ms effective)	Fixed (Exactly 50.0 ms)	High (≈ 128 ms centroid)	**Ultra-Fast** (0.5 ms group delay)
Primary Recommended Use Case	**Primary VR Gaming Mode** (Best balance of speed & sharpness)	**Biomechanical Research** (When wave shape fidelity is sacred)	**Diagnostic Lab Tool** (Direct spectral power visibility)	**Reflex / Onset Trigger** (Instantaneous twitch detection)

13.4 Concluding Engineering Insights & Future Research Horizons

The IIT Gandhinagar SSRP Project 7 has demonstrated that bridging biological human muscle contraction to virtual reality does not require million-dollar medical laboratory equipment or inscrutable black-box neural networks. By applying the timeless principles of **Linear Systems and Signals (as codified by B.P. Lathi)**—continuous-time electrophysiology, discrete-time sampling, Butterworth pole placement, Hamming window truncation, Fourier spectral duality, and causal difference equations—we can construct robust, reliable, and deeply intuitive cybernetic interfaces.

> 🔬 **Technical Fact & Definition: Key Lessons for Future Cybernetic Engineers**
>
> 1. **Physics First, Math Second:** Never attempt to filter a signal until you understand its physical origin. Solving the 6 kg weightlifting paradox required recognizing that isometric contraction is real biological electricity, not electrical noise.
>
> 2. **Causality is King in Interactive Systems:** Elegant mathematical tools that require future data (like `filtfilt`) are useless in real-time computing. True engineering mastery lies in managing causal initial conditions and state vectors (z_i) across streaming packet boundaries.
>
> 3. **The Power of Parallel Architecture:** Running multiple filter models concurrently in memory eliminates mode-switching latency, providing a clean testbed for empirical scientific comparison.

Future Horizons for SSRP Project 7:

Building upon the solid mathematical foundation established in this document, future research teams at IIT Gandhinagar will explore three advanced technological frontiers:

- **Adaptive LMS Noise Cancellation:** Implementing an adaptive Least Mean Squares (LMS) filter that uses an auxiliary antenna channel to continuously model and subtract fluctuating power-line harmonics in real time.
- **High-Density Multichannel sEMG Arrays:** Replacing the single-channel bipolar electrode with a 64-channel flexible grid array to decode individual motor unit action potential velocities across the biceps brachii.

- **Closed-Loop Functional Electrical Stimulation (FES):** Adding bidirectional haptic feedback by sending low-current electrical stimulation pulses back into the user's triceps, allowing the user to literally *feel* the physical weight of the virtual dumbbell in VR!

END OF COMPREHENSIVE TECHNICAL REFERENCE MANUAL

IIT Gandhinagar SSRP 2026 • Project 7 • Real-Time EMG Signal Processing & VR Weightlifting